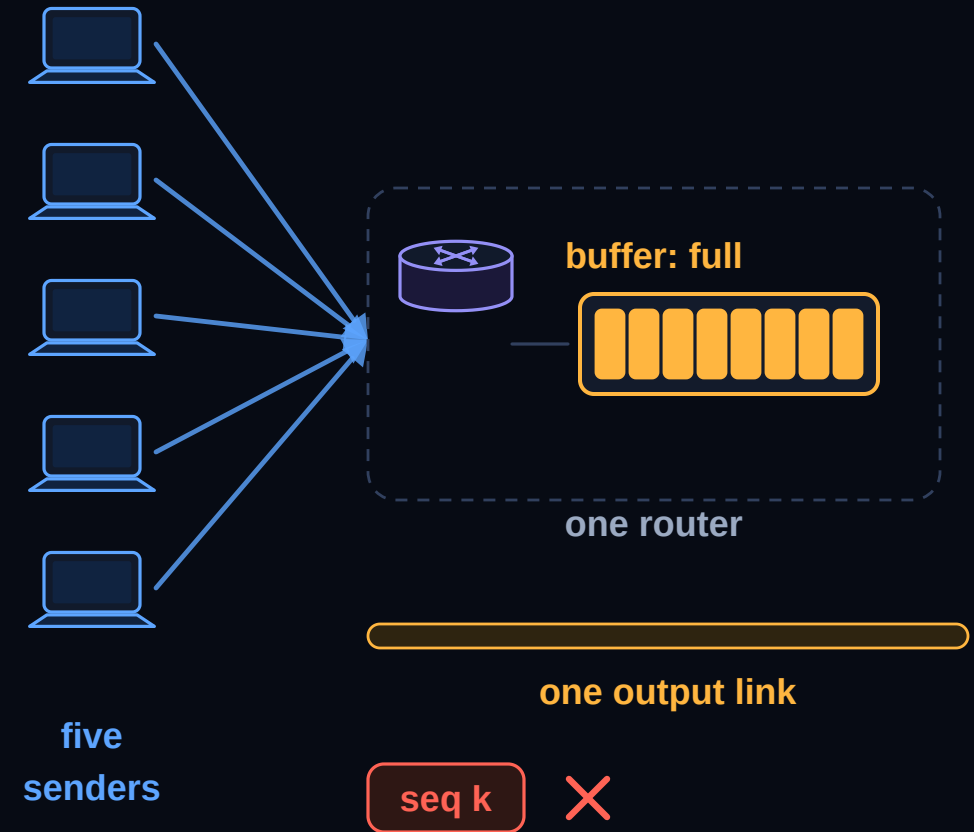


Congestion Control

How fast should a sender send?

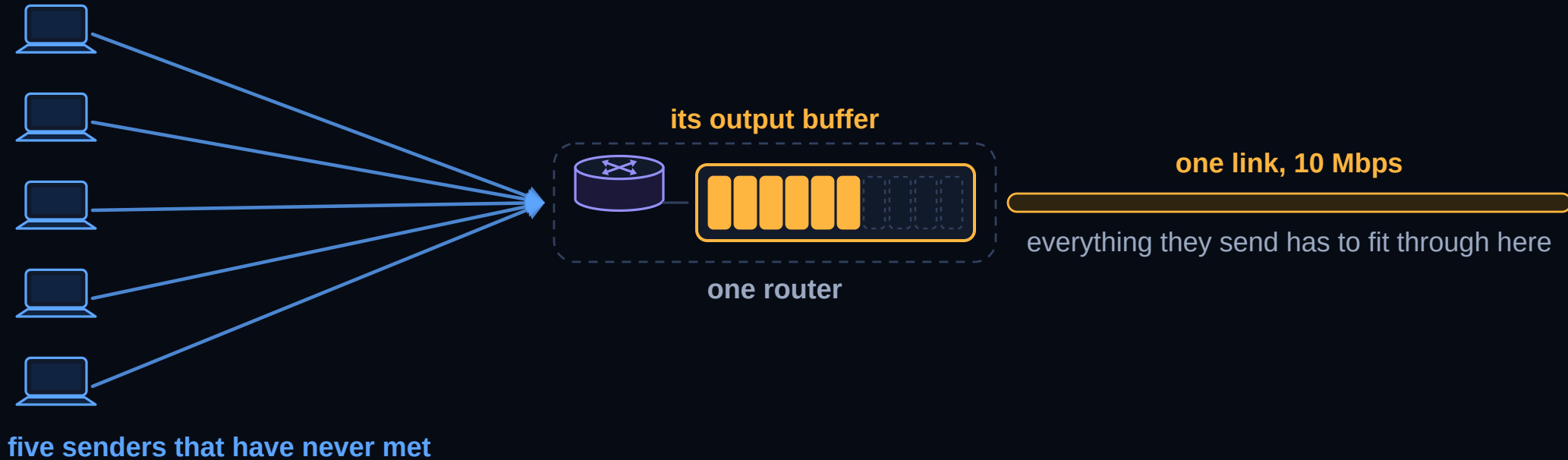
Every link in the internet is shared and nobody hands out the shares. This lecture shows what goes wrong when senders get their rate wrong, how a sender can find out, and the rule that has made every sender limit its own rate since 1988.

Yaxiong Xie



nobody told any of them how fast to send

Who decides how much of a link each sender gets?



no reservation is made



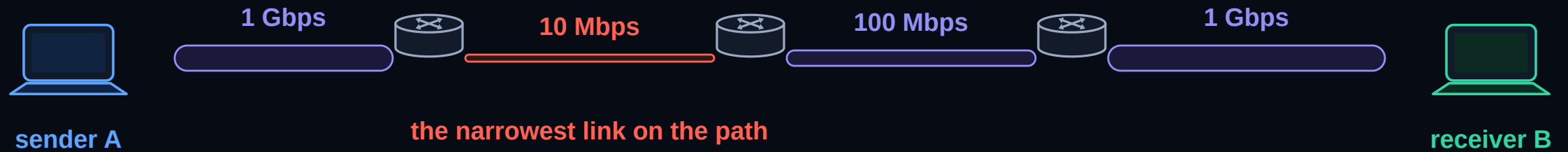
no admission control



no router hands out shares

Nobody. A sender may put packets on the internet at any rate, at any moment.

So how fast should a sender send?

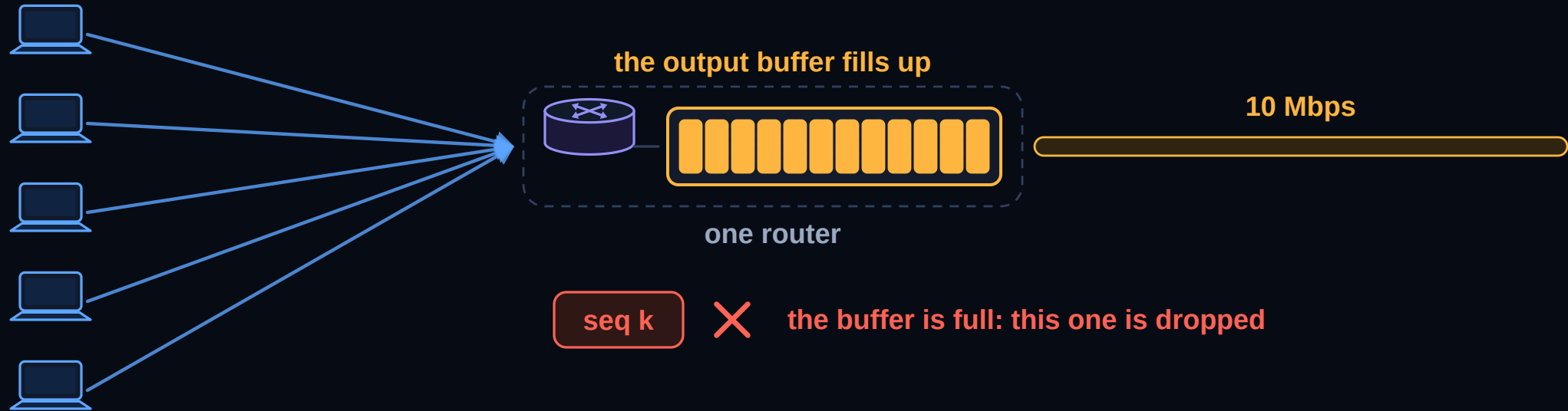


✗ as fast as the application produces data
the application has no idea what the path can carry

✗ as fast as sender A's own card can go: 1 Gbps
a hundred times what the 10 Mbps link can take

The right rate is set by the slowest link and by who else uses it. A sees neither.

What happens when senders get it wrong?



⌚ First it delays

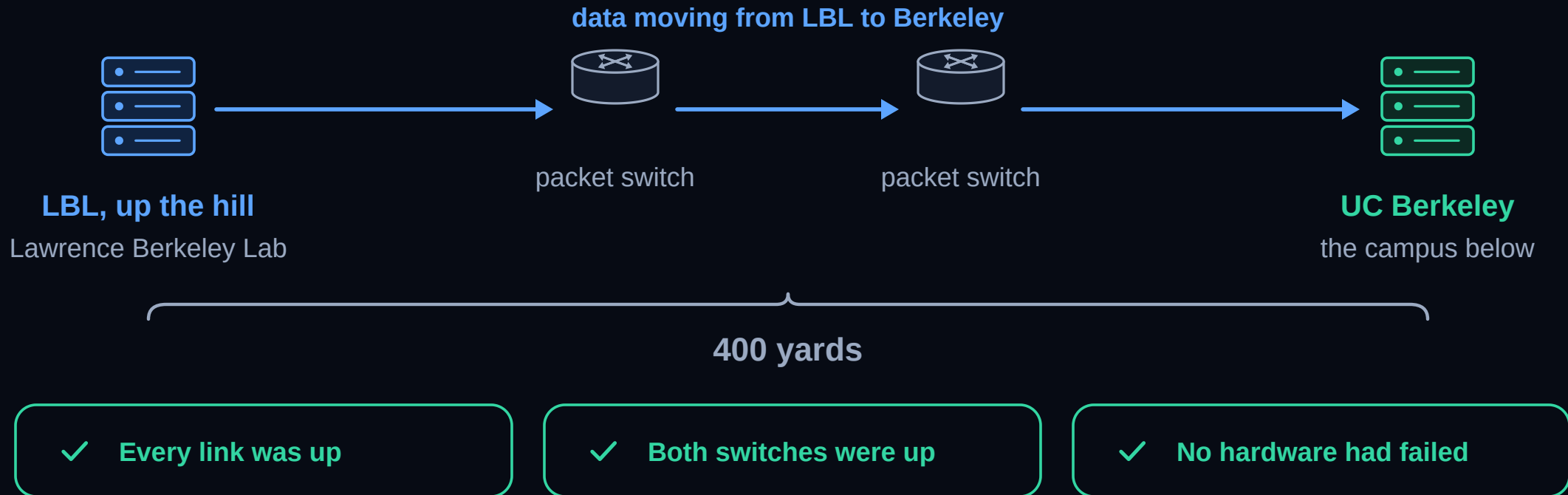
a packet that fits waits behind everything already in the buffer

🗑 Then it throws away

when the buffer is full there is nowhere left to put a packet

The network has two answers to too much traffic: delay it, then throw it away.

October 1986: two sites, 400 yards apart, two hops



Van Jacobson called it the first of what became a series of “congestion collapses” (SIGCOMM 1988).

Every link and every packet switch was working exactly as it was designed to.

The same path, 800 times slower

THE THROUGHPUT FROM LBL TO UC BERKELEY

normally

32 kbps

= 4,000 bytes per second

a 1 MB file would take

just over 4 minutes



October 1986

40 bps

= 5 bytes per second

a 1 MB file would take

2.3 days

800 times slower, with the same hardware on the same path

This is what a network does when every sender sends as fast as it can.

Why did the throughput fall instead of levelling off?



rises

more in, more out

flattens

the link is full

falls

more in, less out

Rising and flattening are not surprising: a link has a capacity. Falling is. Part 4 takes the falling part apart.

Past capacity, every extra packet offered reduces what the network delivers.

What did the internet fix after 1986?



No reservation protocol

was ever deployed on
the public internet



No router was redesigned

a full buffer still means
a packet is dropped



Nobody in charge

no authority hands out
any sender's share

The 1986 collapse could happen again tomorrow.

Every condition that produced it is still in place: shared links, no reservations,
no admission control, and buffers that still overflow.

None of it. The public internet still reserves nothing and refuses no one.

Why doesn't that happen every day?

What changed in 1988 was only this:

**every sender limits its own rate,
and keeps adjusting it, forever**

no message from the network, no coordination between senders

in the hosts, not the network

one algorithm, added to TCP
in every operating system

nothing had to be agreed

each host could adopt it on
its own, one machine at a time

and the idea still runs

on the machine in front of you,
though the exact rule has changed

Congestion control is a rule the end hosts keep. This lecture is that rule.

Three questions, ten parts

01



What exactly goes wrong

inside the network when
senders send too much?

1 Congestion

2 Causes

3 Challenges

4 The costs

02



How can a sender control

its rate, and find the right
one when nothing tells it?

5 The rate

6 Probing

03



How should the rate move

up and down, so that senders
share a link fairly?

7 AIMD

8 Throughput

9 Slow start

10 On a loss

Nothing in this lecture needs a change to any router. It is all in the sender.

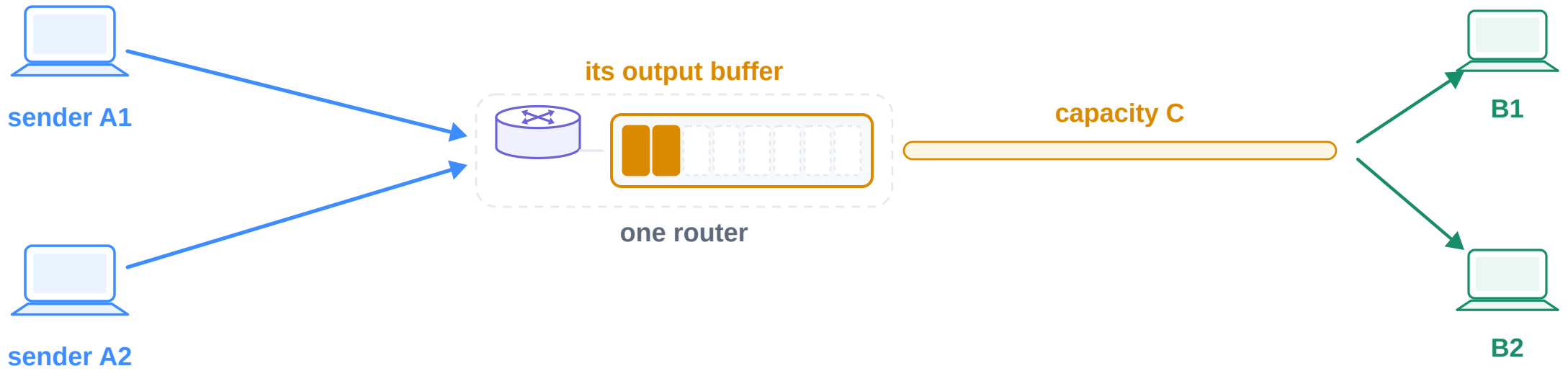
What is congestion?

01

What congestion is

Two senders share one link. When together they send more than it can carry, the router's buffer takes the difference, fills up, and then drops packets. This part watches that happen, one step at a time, and then names it.

The model: two senders, one router, one output link



? They never meet

A1 cannot measure A2, and neither of them can see the buffer

🗄️ They share one link

every packet either of them sends goes through the same buffer

The smallest picture in which congestion can happen at all.

Nothing in this picture says how fast either sender is allowed to send.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

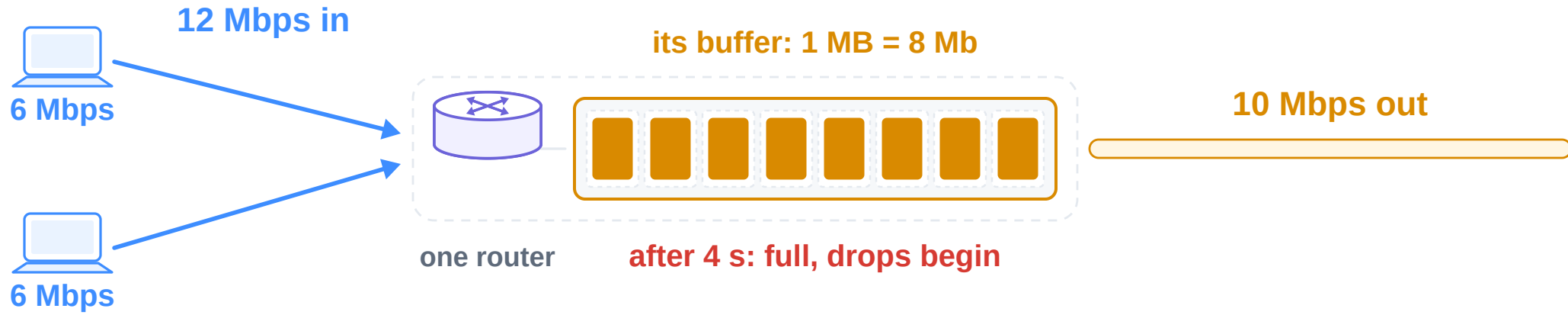
8 Throughput

9 Slow start

10 On a loss

• ——— What congestion is

12 Mbps in, 10 Mbps out. Where does the rest go?



12 Mbps arrive, but only 10 Mbps can leave.

every second: 12 Mb in and 10 Mb out, so 2 Mb stay in the buffer, until it is full



For 4 s the buffer absorbs it

nothing is lost yet, but waits keep growing



Then it is full

the 2 Mb a second the link cannot send are dropped

Into the buffer, 2 Mb more each second, until it is full. Then the extra is dropped.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

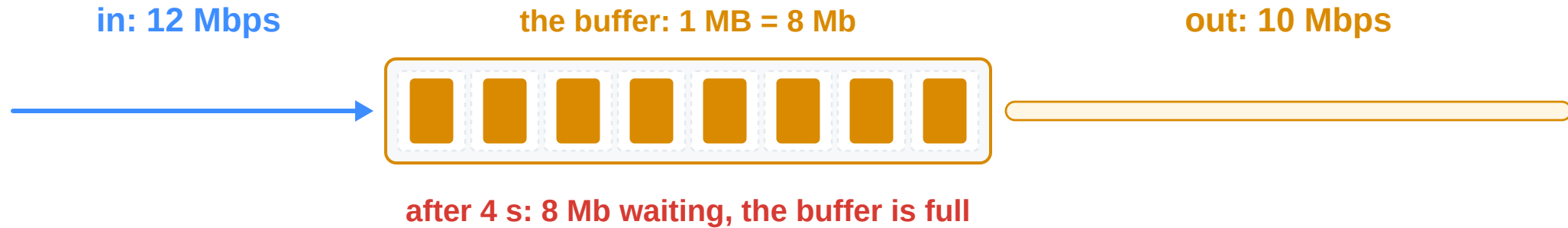
8 Throughput

9 Slow start

10 On a loss

- ——— What congestion is

How long does 2 Mbps take to fill a 1 MB buffer?



fill rate $= \text{in} - \text{out} = 12 - 10 = 2 \text{ Mbps}$

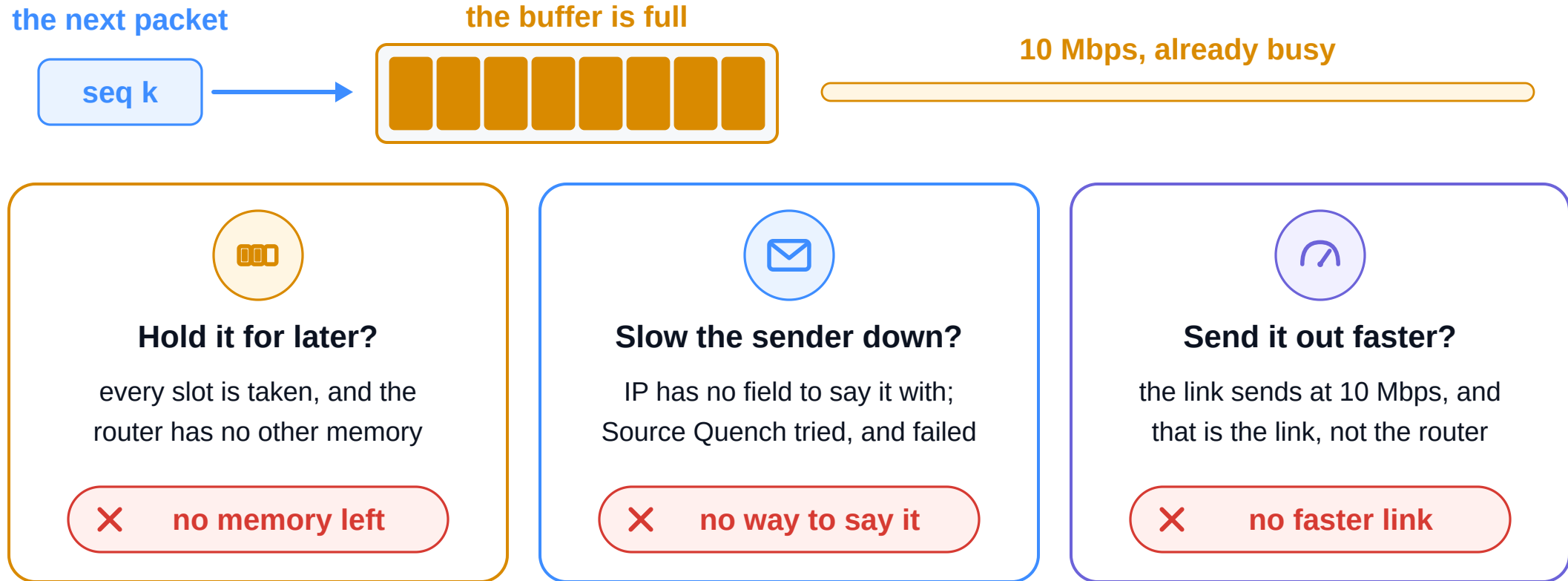
fill time $= \text{size} \div \text{fill rate} = 8 \text{ Mb} \div 2 \text{ Mbps} = 4 \text{ s}$

An 8 MB buffer (64 Mb) instead: $64 \div 2 = 32 \text{ s}$. The same end, 28 seconds later.

Four seconds. A bigger buffer changes when that happens, not whether it happens.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

The buffer is full. What could the router do?



A router has no memory to spare, no field to speak through, and no say over the link rate.

• ——— What congestion is

So the router throws the packet away

the next packet

seq k



the buffer is full



10 Mbps, already busy



It discards the packet.

a tail drop: what every router does
by default when a buffer is full

And says nothing about it.

no message goes back to the sender;
only its own drop counter changes

The sender finds out only because an ACK never comes.

In classic TCP that silence is the only signal a sender gets, and Part 6 is built on it.

With nowhere to put it and no way to say so, the router drops it in silence.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

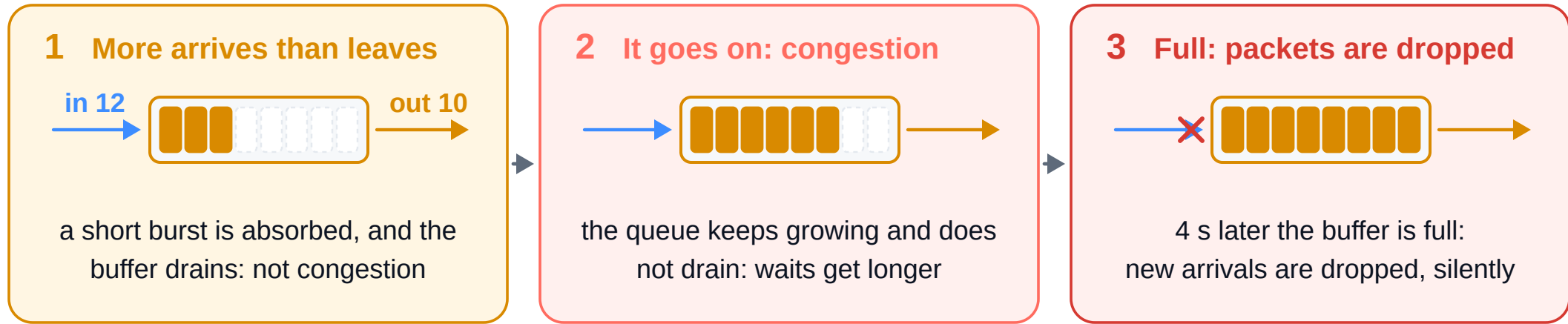
7 AIMD

8 Throughput

9 Slow start

10 On a loss

So what is congestion?



**Congestion: more arrives at a link than it can send,
for long enough that a queue builds and does not drain.**

Delay grows first; once the buffer is full, packets are dropped.

Nothing is broken, just as in 1986: every link and router is doing its job. There is simply more traffic than the link.

Congestion is a queue that does not drain: delay comes first, drops come last.

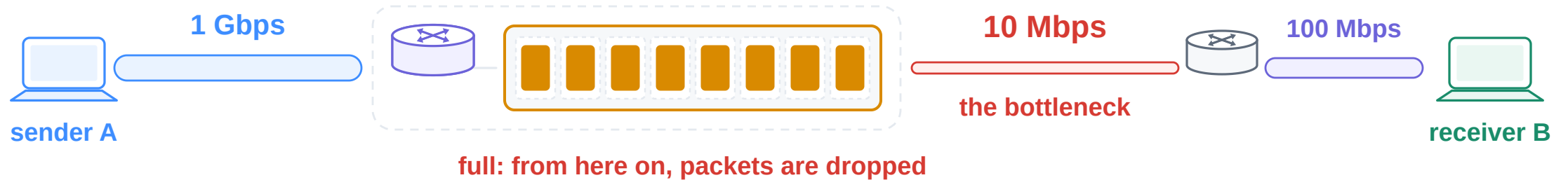
Why does congestion happen?

02

Why congestion happens

Part 1 watched a buffer fill because more arrived than its link could send. This part asks how a network gets there: one sender faster than its path, or many senders that are each slow enough and together too fast.

Can one sender congest a path on its own?



A cannot see the 10 Mbps link. Its own card runs at 1 Gbps, so that is how fast it sends.

fill time = $8 \text{ Mb} \div (1,000 - 10) \text{ Mbps} \approx 8 \text{ ms}$
slide 14's arithmetic: a 1 MB buffer lasts eight milliseconds

Yes. A sender that cannot see the bottleneck overruns it on its own, in milliseconds.

Can senders that each stay below the link still congest it?



✓ each: 4 Mbps, well below 10
no sender does anything wrong

✗ together: $4 + 4 + 4 = 12$ Mbps
Part 1's two senders were this case: $6 + 6$

Yes. Every sender can be well below the link, and all of them together still overrun it.

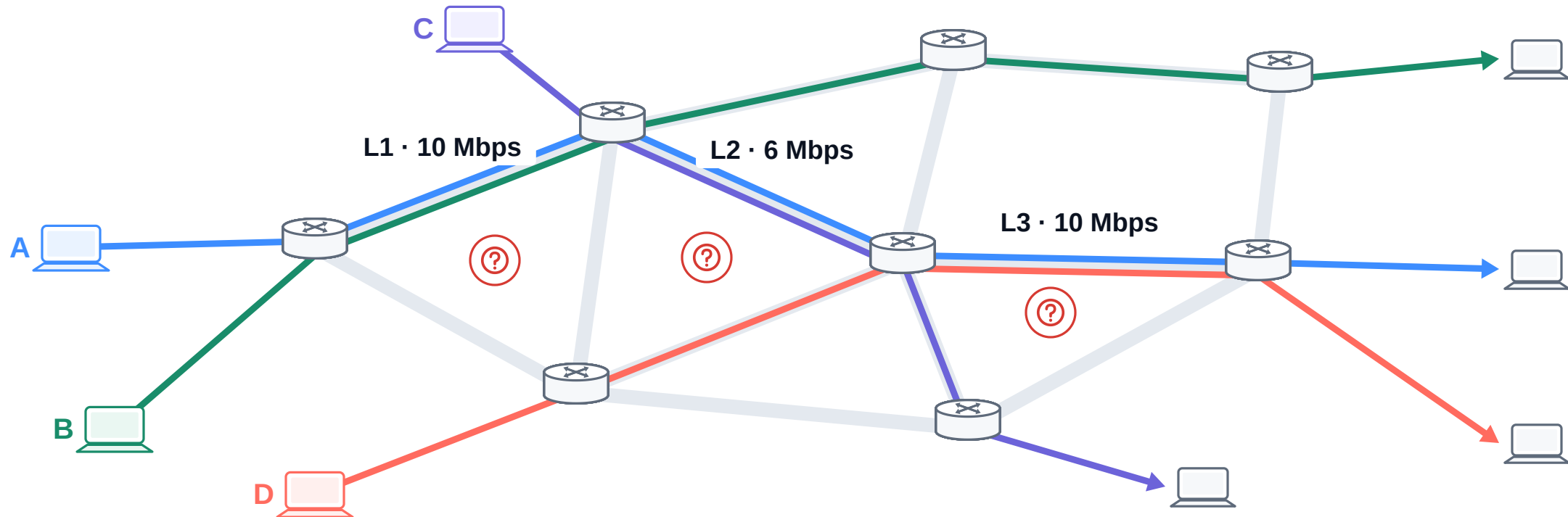
Why is congestion control challenging?

03

Why control is hard

Part 2 showed how senders overrun a link. Stopping that is harder than it sounds: the internet is a mesh, every flow shares links with flows it cannot see, the senders cannot coordinate, and even a rate that is right now does not stay right.

The internet is a mesh: who shares which link?

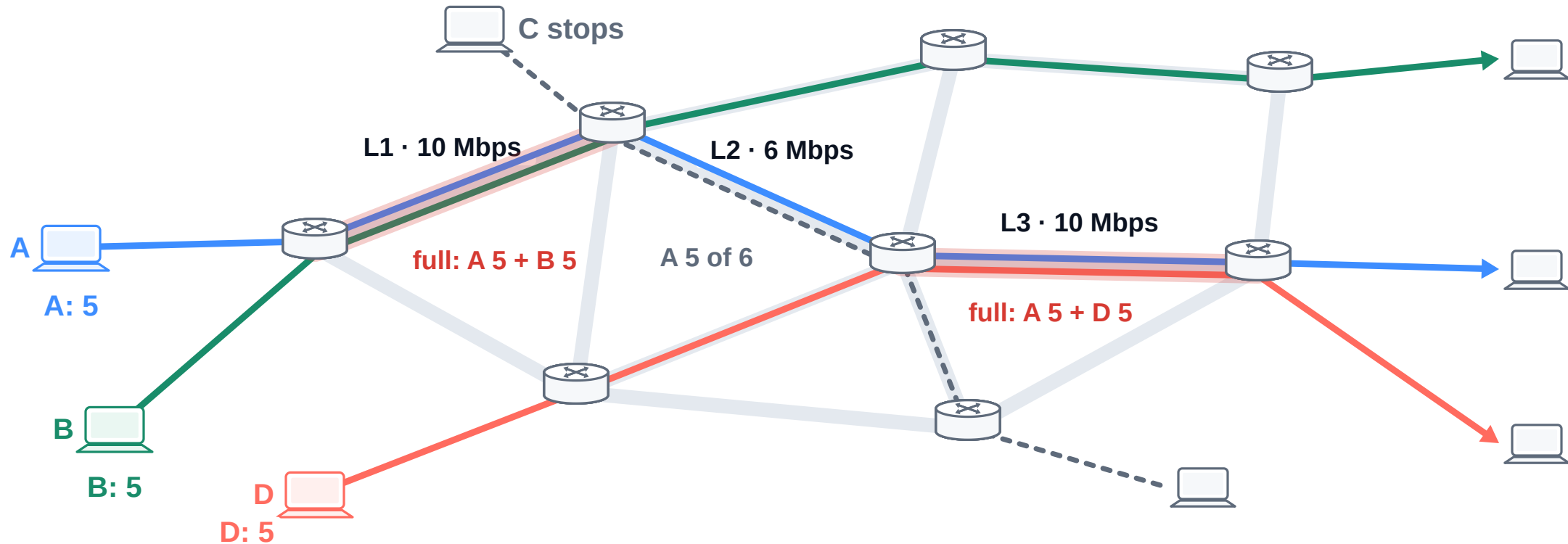


So B, C and D can each slow A down, on three different links.

When A's packets wait or get dropped, A cannot tell which flow is the cause.

A shares links with B, C and D: any of them can slow A down, and A cannot tell which.

How can a flow that B never meets change B's share?



C stopped, and B fell from 7 to 5, though B never shares a link with C.

Through A: flows are chained by shared links, so one change moves every share.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

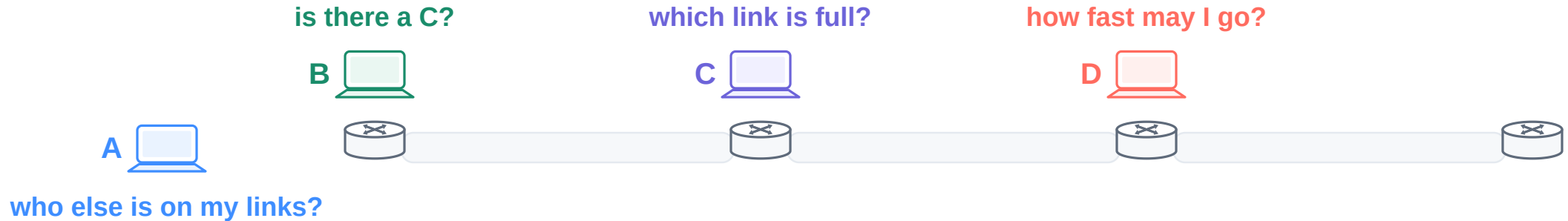
7 AIMD

8 Throughput

9 Slow start

10 On a loss

Could the senders simply agree on a split?



❓ **They cannot see**
none knows who shares which
link, or which link is full

✗ **They cannot talk**
nothing carries messages
from one sender to another

🚫 **Nobody is in charge**
no router and no authority
hands out shares (slide 2)

Agreeing would take a view of the whole network and a way to talk. There is neither.

No. Each sender has to find its rate alone, from what its own packets and ACKs tell it.

Once every sender has the right rate, are we done?

+ Competition

other flows start and stop,
so each fair share moves
(next slide)

↔ Mobility

a phone moves: its link changes
speed, and its path can change
(the slide after)

↻ And more

routes change and links fail;
traffic comes in bursts;
each moves the right rate

The right rate is a moving target.

A sender cannot find it once and keep it: it has to keep finding it, alone.

No. Other flows come and go and devices move, so the right rate keeps moving.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

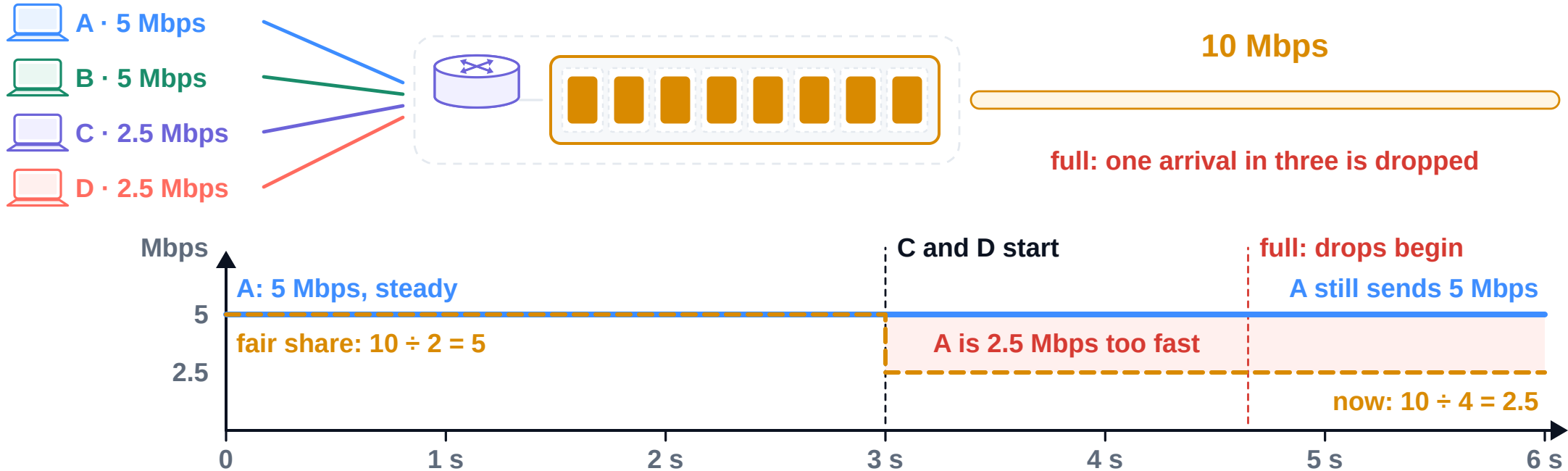
7 AIMD

8 Throughput

9 Slow start

10 On a loss

Competition: what happens when two more flows start?

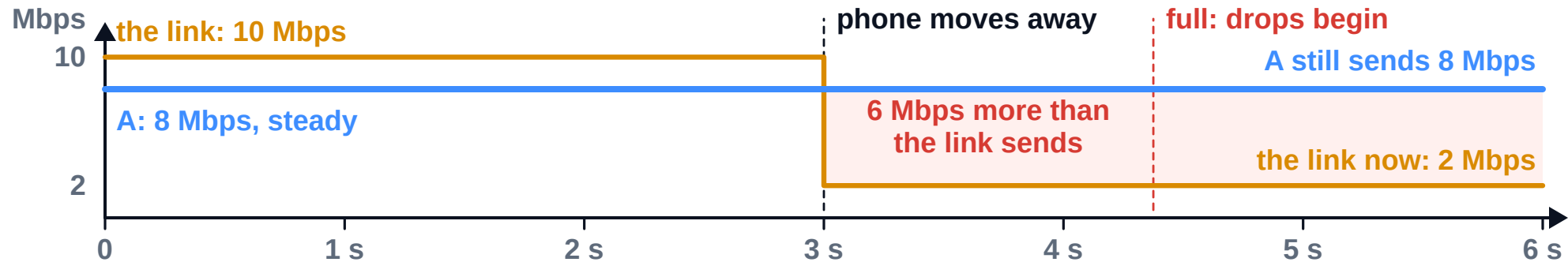
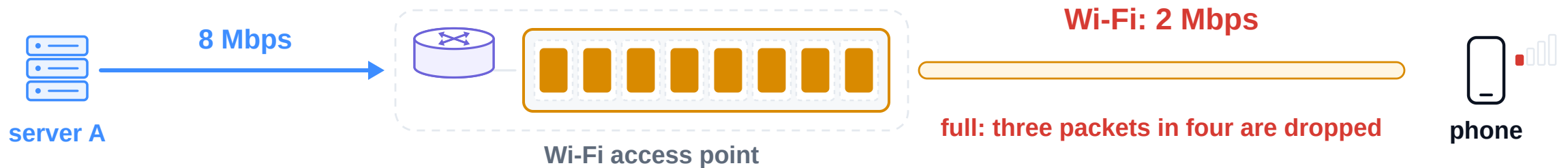


A changed nothing. Its right rate fell from 5 to 2.5 because C and D started.

Nothing tells A that. It finds out only when its packets wait and are dropped.

A's right rate falls when other flows start, and nothing in the network tells A.

Mobility: what if the link itself slows down?



A changed nothing. The link under it fell from 10 to 2 Mbps.

Wireless links change speed all the time: phones move, walls block the signal, other devices share the air.

The right rate moves even with no other flows: the link itself changes speed.

So why is congestion control hard?



A mesh of shared links

each flow shares some links with flows it cannot see



Hidden causes

a sender cannot tell which flow is slowing it down



No coordination

the senders can neither see the network nor talk



Competition

flows start and stop, so every fair share moves



Mobility

devices move, so links change speed

Each sender has to find its rate alone, from its own packets, and keep finding it.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

8 Throughput

9 Slow start

10 On a loss

What does congestion cost?

04

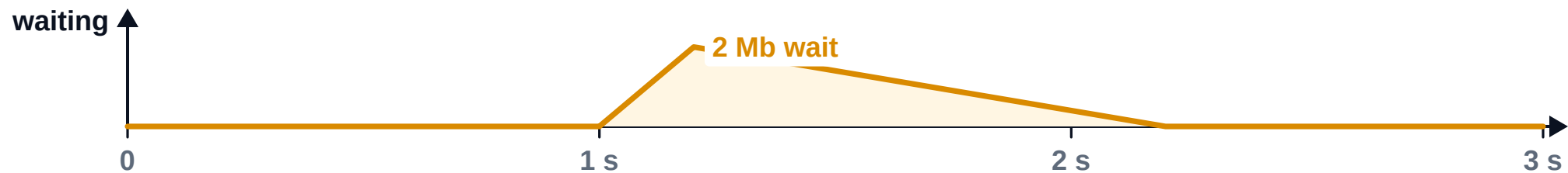
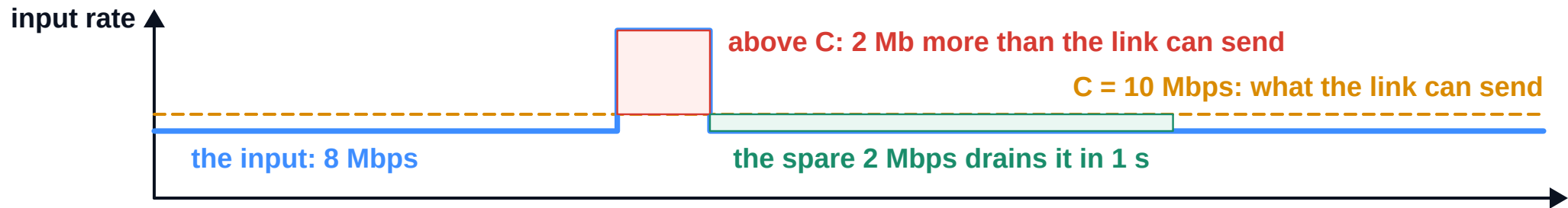
What congestion costs

Parts 1 to 3 showed what congestion is, why it happens and why it is hard to control. This part counts what it costs: four costs, each a thing the network spends capacity on and gets nothing back for. Together they explain why the curve on slide 7 falls.

Cost 1: below capacity, why does a packet ever wait?

COST 1 OF 4

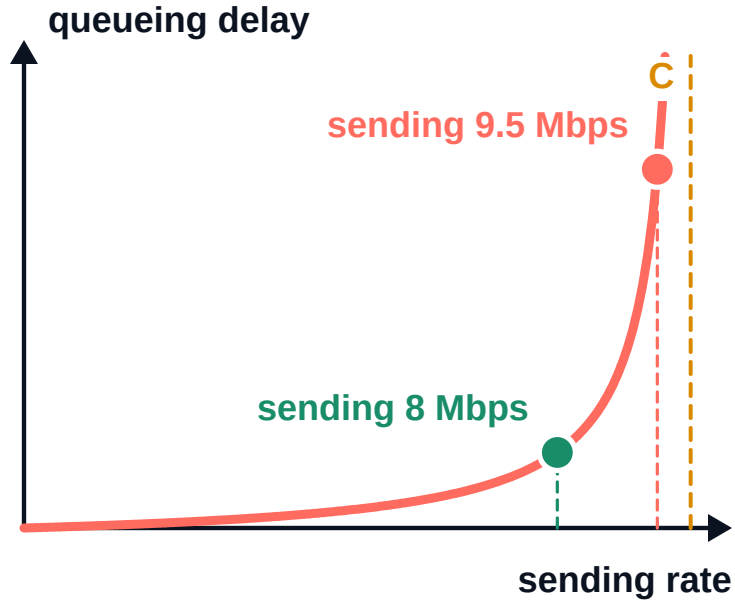
sending about 8 Mbps



On average the link keeps up; during a burst it does not, and the extra packets wait.

Cost 1: why does the wait grow near capacity?

COST 1 OF 4



Below C nothing is lost, only delayed.
For a video call, the delay is the cost.

sending 8 Mbps

spare 2 Mbps · waits: none, then 0.4 s

rate

waiting

sending 9.5 Mbps

spare 0.5 Mbps · waits: 0.8 s, then 2.8 s

rate

waiting

Near capacity a smaller burst starts a queue, and every queue drains more slowly.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

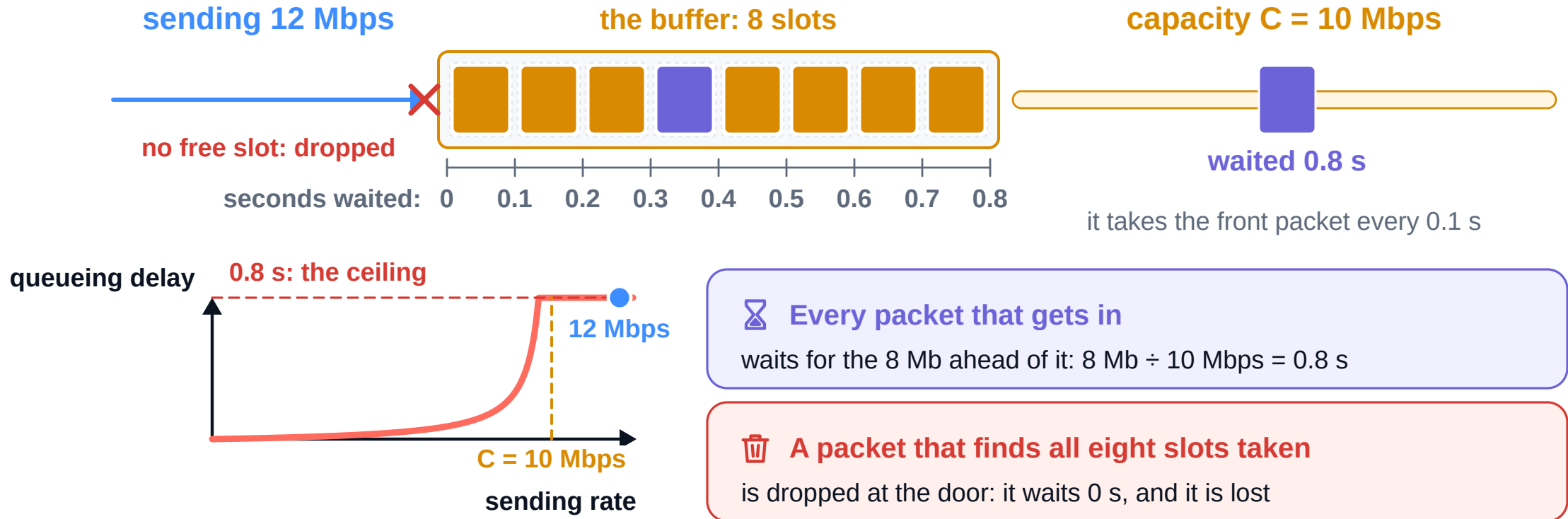
8 Throughput

9 Slow start

10 On a loss

Cost 1: once the buffer is full, every packet waits 0.8 s

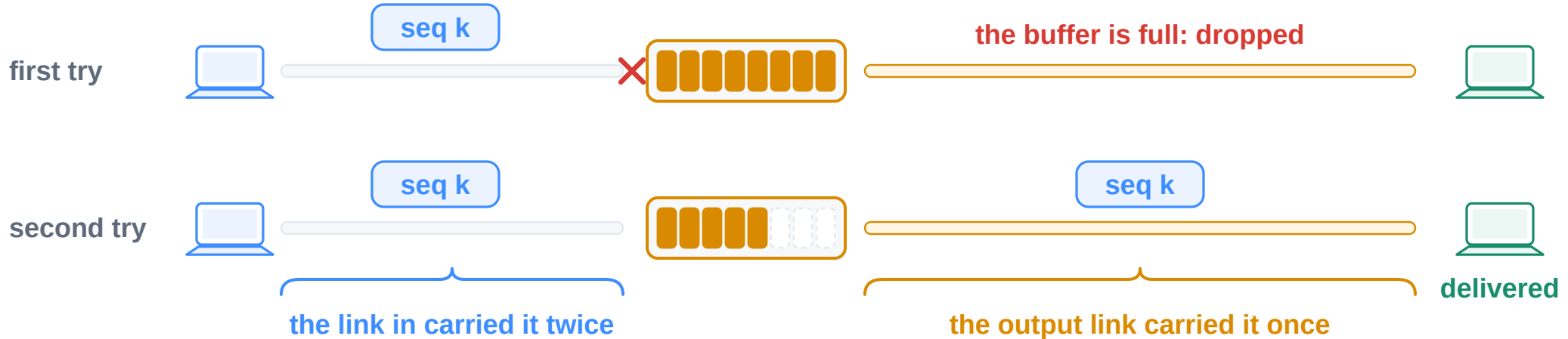
COST 1 OF 4



A full buffer makes every packet it takes wait 0.8 s, and drops only those it has no room for.

Cost 2: a dropped packet has to be sent again

COST 2 OF 4



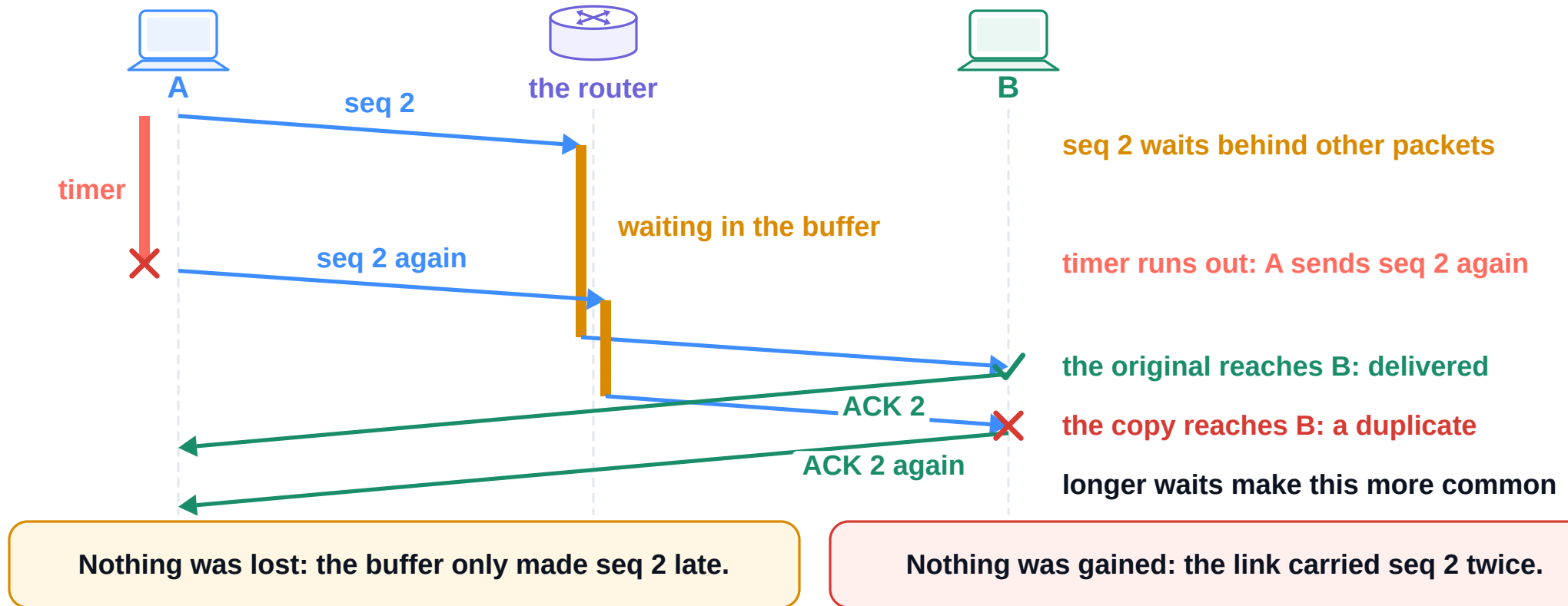
The dropped copy never reached the output link.

The sender and every link before the drop pay for the packet twice.
Resending it takes nothing from the output link: it crosses that link once.

A drop costs every link before it a second copy; the links after it pay once.

Cost 3: resending a packet that was never lost

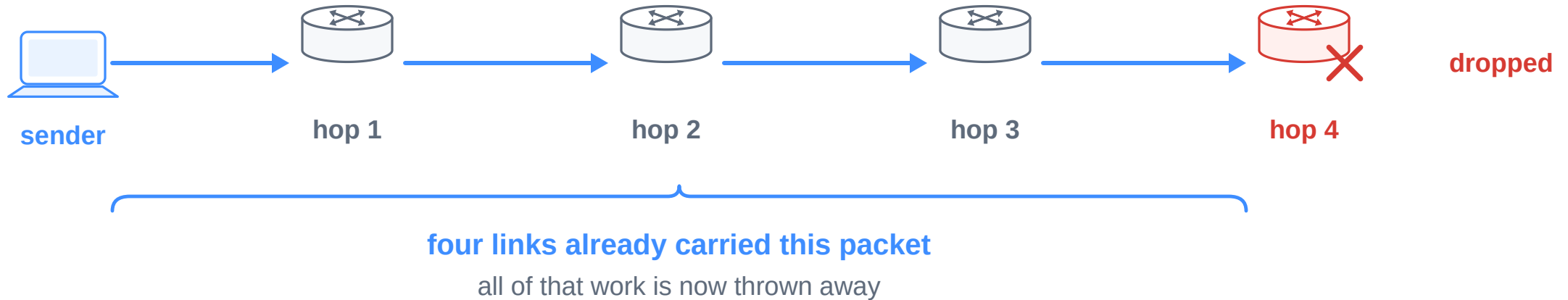
COST 3 OF 4



Pure waste: the link carries a second copy of data that had already arrived.

Cost 4: a dropped packet wasted every link it already crossed

COST 4 OF 4

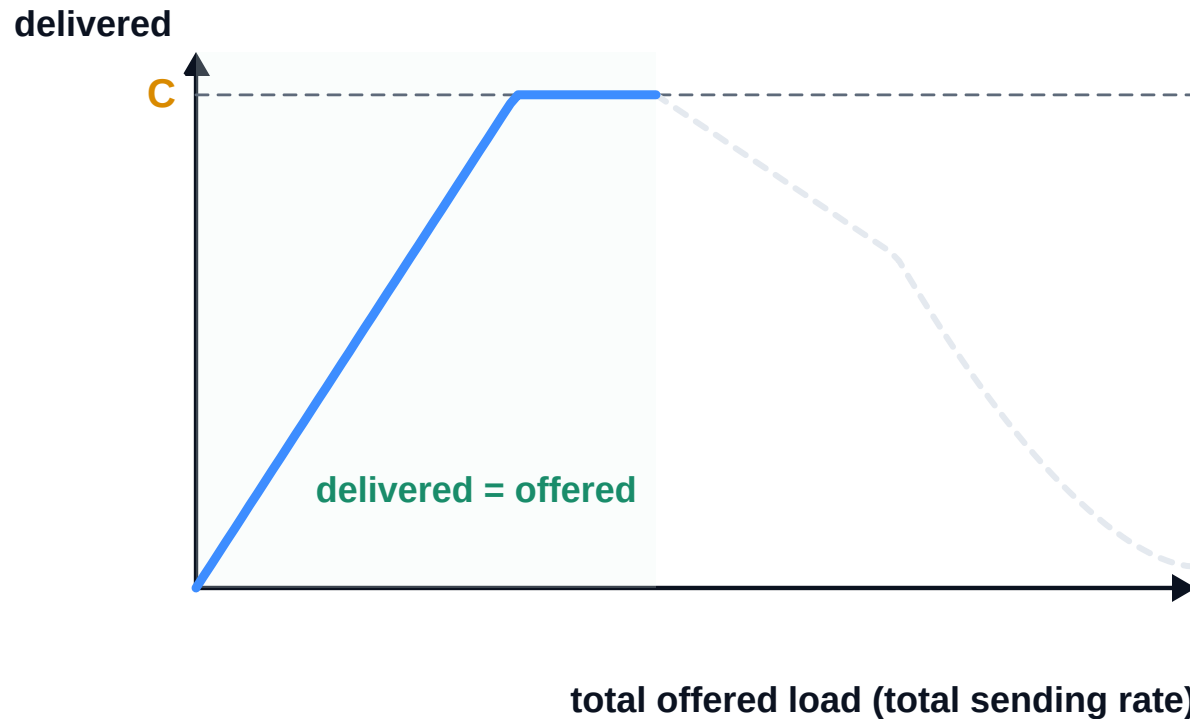


Those four links could have carried another connection's packet, which would have arrived.

Cost 2 was one extra copy on the links before a drop. On a long path those links are shared, so every connection on them pays, and the cost grows with how far the packet got.

The later a packet is dropped, the more of the network's work it takes with it.

Why does the curve rise, and why does it stop?



✓ Below C it rises

everything offered is delivered: the buffer drains as fast as it fills, so nothing is dropped and nothing has to be sent twice

🗄 At C it stops rising

the link transmits C bits every second and not one more, whatever arrives at it.
 C is a ceiling on what it can deliver

Up to C , delivered equals offered. C is a hard ceiling on any link.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

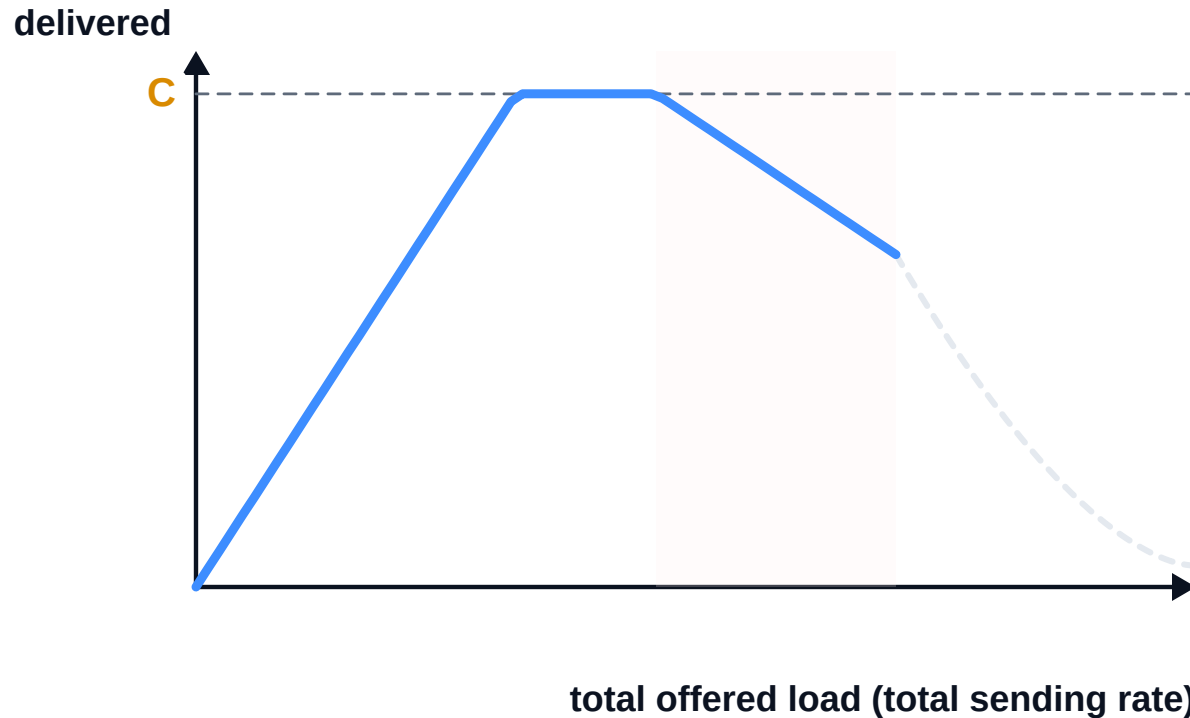
7 AIMD

8 Throughput

9 Slow start

10 On a loss

Why does it fall below the ceiling?



goodput: the green part of the 10 Mbps link

no repeats



$$\text{goodput} = 10 \times (1 - 0) = 10 \text{ Mbps}$$

20% repeats



$$\text{goodput} = 10 \times (1 - 0.2) = 8 \text{ Mbps}$$

50% repeats



$$\text{goodput} = 10 \times (1 - 0.5) = 5 \text{ Mbps}$$

☐ new data ☐ data it carried before

A packet dropped at this buffer never used it.

The repeats are cost 3: a timeout on a packet that was only late, or a window resent whole.

The link is still full. A growing share of what it carries is data it carried before.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

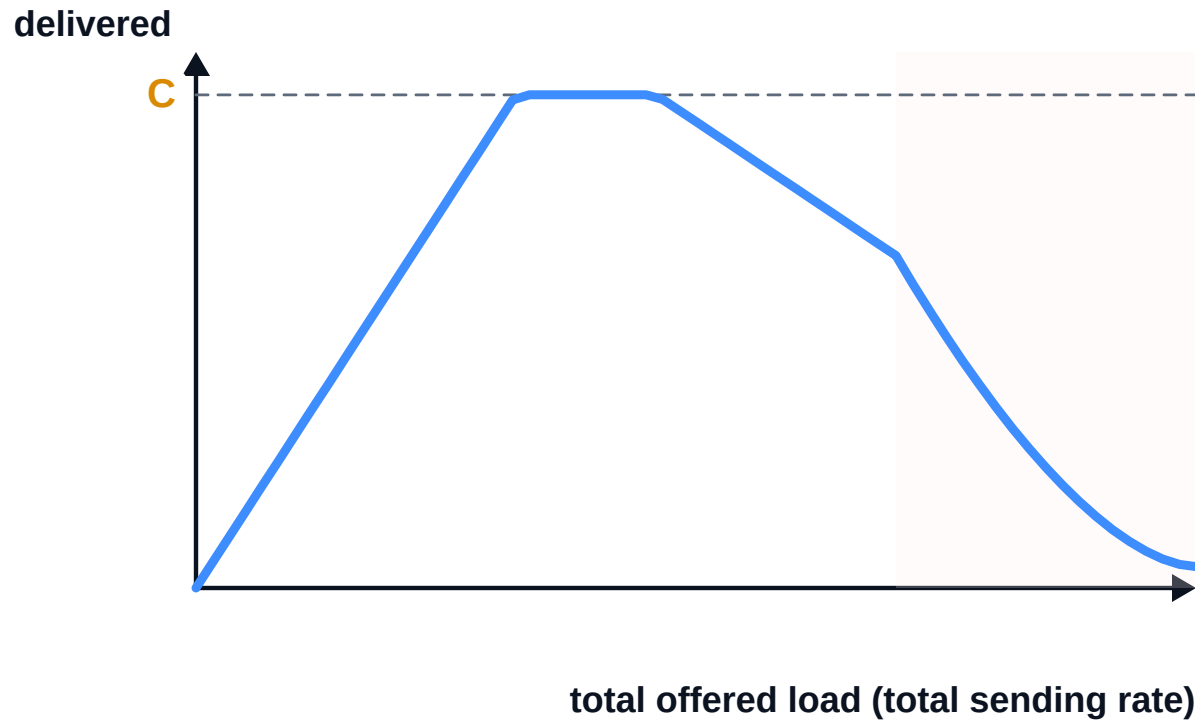
7 AIMD

8 Throughput

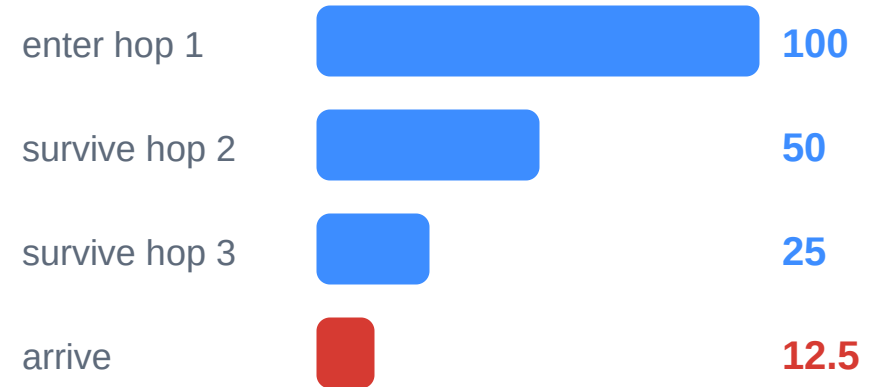
9 Slow start

10 On a loss

Why does it keep falling instead of settling?



four hops; hops 2, 3 and 4 each drop half



**Hop 1 did 100 packets of work.
Only 12.5 of them arrived.**

The other 87.5 took capacity from
packets that would have arrived.

The waste compounds hop by hop, so the delivered share collapses instead of sagging.

Can one sender fix this on its own?



It cannot see the cause

the traffic filling the buffer
comes from senders this one
has never heard of



It is not the only victim

the packets that get dropped
belong to connections that
did nothing wrong



Slowing down alone loses

a sender that backs off by itself
just hands its share to the
ones that did not

So the fix cannot be a rule that one sender keeps.

It has to be a rule that every sender keeps, which is what the rest of the lecture builds.

No. Congestion is the first problem in this course that one sender cannot solve.

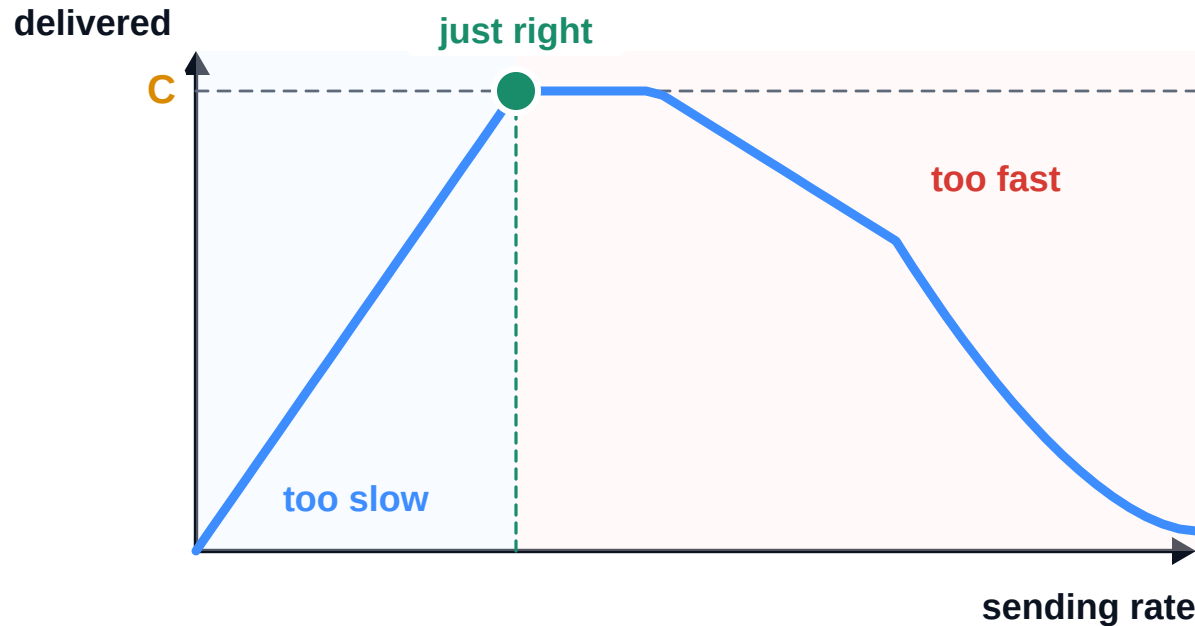
What is congestion control?

05

Control the sending rate

Parts 1 to 4 showed what congestion is, why it happens, why it is hard to control and what it costs. Congestion control is the sender's answer: keep its sending rate at what the path can carry, no faster and no slower. This part says what that rate is, and gives the sender a way to set it.

Where on Part 4's curve should a sender be?



⤿ Too slow

the link has room to spare:
capacity that nobody uses

✓ Just right: the knee

the sending rate equals C :
the link is full, nothing waits

⚠ Too fast

the buffer fills, packets drop,
repeats pile up: delivered falls

Part 4's curve, with one sender: the load it offers is its own sending rate.

At the knee: fast enough to fill the link, no faster. That is congestion control.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

8 Throughput

9 Slow start

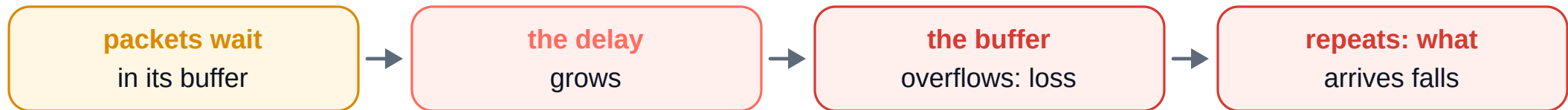
10 On a loss

What rate fills the path without overflowing it?



Send at 10 Mbps: the bottleneck is exactly full, and nothing waits.

Send faster than 10 Mbps, and Part 4's costs follow, in this order:



The bottleneck's rate. Faster than that, packets only wait, then drop.

Two questions every sender has to answer

01



**How can a sender
control its rate?**

what it can change, and how that sets the rate

Part 5 answers it

02



**What rate should
it aim for?**

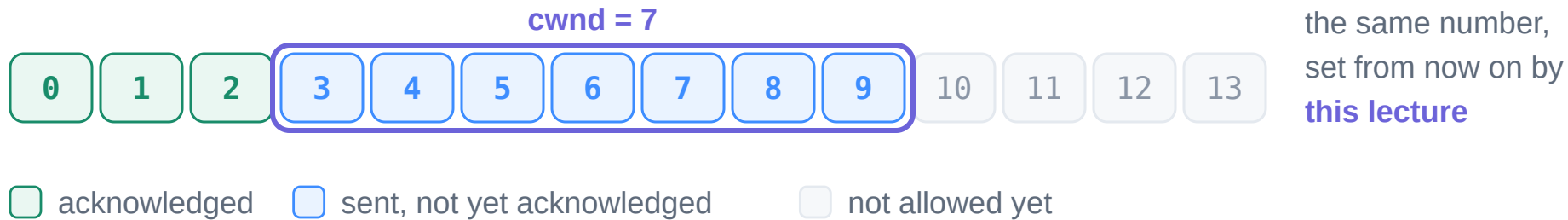
the bottleneck's, which nothing on the path tells it

Parts 6 and 7 answer it

First the knob, then where to set it.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

How can a sender control its sending rate?



the same number,
set from now on by
[this lecture](#)

How W was set before

a fixed number, chosen once so that
the pipe would stay full

it never moved while the
connection was running

How $cnwnd$ is set from here

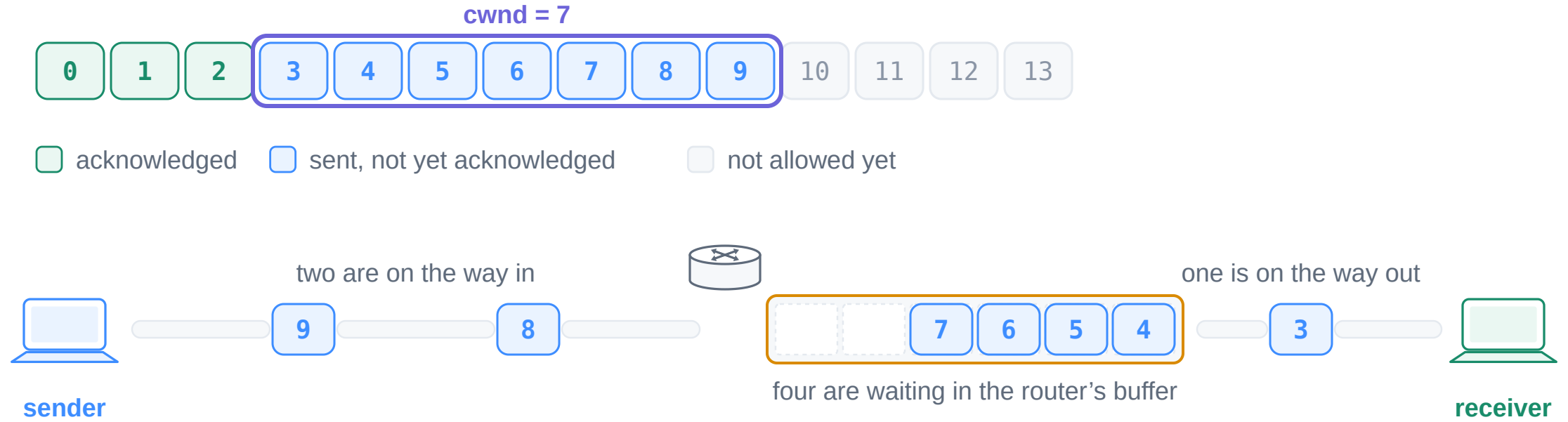
the sender recomputes it constantly,
from what the ACKs tell it

it moves on every ACK and
on every loss, for ever

Through its sliding window, now called $cnwnd$. Only how the window is chosen changes.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

The blue slots are packets that are inside the network

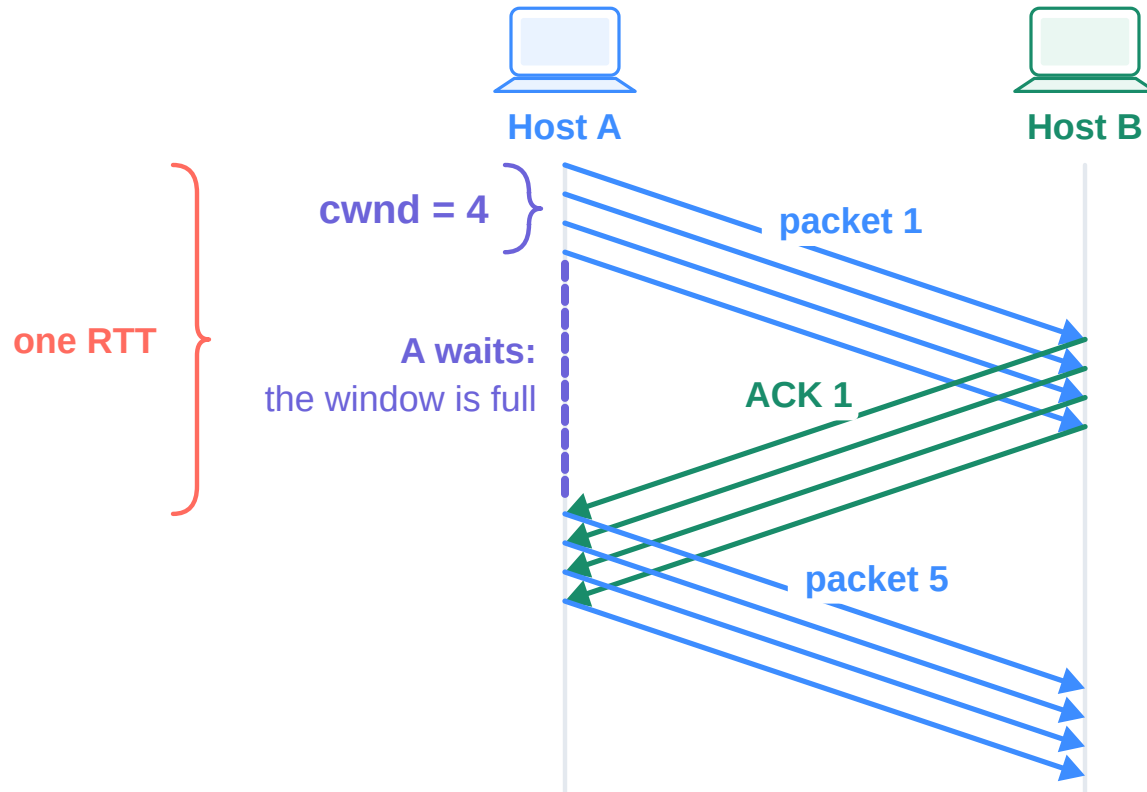


Seven blue slots, seven packets inside the network. cwnd is that count.

A packet sent and not yet acknowledged is a packet inside the network right now.

- 1 Congestion
- 2 Causes
- 3 Challenges
- 4 The costs
- 5 The rate
- 6 Probing
- 7 AIMD
- 8 Throughput
- 9 Slow start
- 10 On a loss

What does a sender do with $cwnd = 4$?



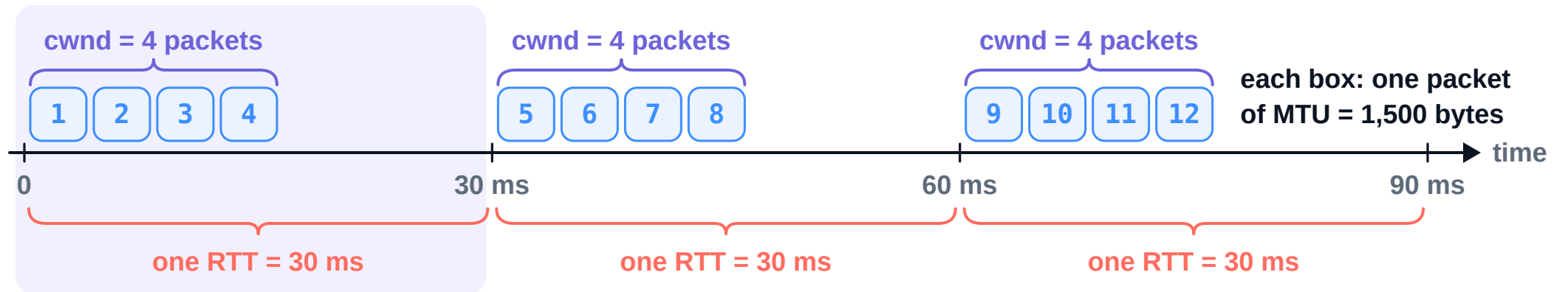
- 1 A sends $cwnd = 4$ packets, one right after another.
- 2 Four are in the network: the window is full, so A waits for an ACK.
- 3 B sends back an ACK for every packet.
- 4 Each ACK frees one slot, and one new packet goes: 5, 6, 7, 8 leave one RTT after 1, 2, 3, 4.

One window leaves every RTT.
The returning ACKs set the pace: the ACK clock.

The window refills only as ACKs return, so one window of $cwnd$ packets leaves per RTT.

- Control the sending rate

Why is the rate $\text{cwnd} \times \text{MTU} / \text{RTT}$?



In one **RTT**, A sends one window: **cwnd** packets.

Each packet is **MTU** bytes, so one **RTT** carries $\text{cwnd} \times \text{MTU}$ bytes.

$$\text{rate} = \text{bytes per RTT} \div \text{RTT} = \text{cwnd} \times \text{MTU} / \text{RTT}$$

$$= 4 \times 1,500 \text{ B} / 0.03 \text{ s} = 200,000 \text{ B/s}, \times 8 \text{ bits per byte} = 1.6 \text{ Mbps}$$

Every RTT one window leaves: **cwnd** packets of **MTU** bytes. So $\text{rate} = \text{cwnd} \times \text{MTU} / \text{RTT}$.

- 1 Congestion
- 2 Causes
- 3 Challenges
- 4 The costs
- 5 The rate
- 6 Probing
- 7 AIMD
- 8 Throughput
- 9 Slow start
- 10 On a loss

Which part of that rate can the sender change?

$$\text{rate} = \text{cwnd} \times \text{MTU} / \text{RTT}$$

✗ MTU

the path picks it
one packet is as big as every
link on the way will carry:
1,500 bytes on Ethernet

✗ RTT

distance and queueing pick it
a sender can measure RTT,
but it cannot make the path
shorter

✓ cwnd

the sender picks it
a number the sender keeps in
its own memory, and is free
to change at any time

“What rate should I aim for?” is now “what should cwnd be?”

Nothing on the path says. Part 6 is how the sender finds out.

Two of the three terms belong to the path. The third is the sender's to set.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

8 Throughput

9 Slow start

10 On a loss

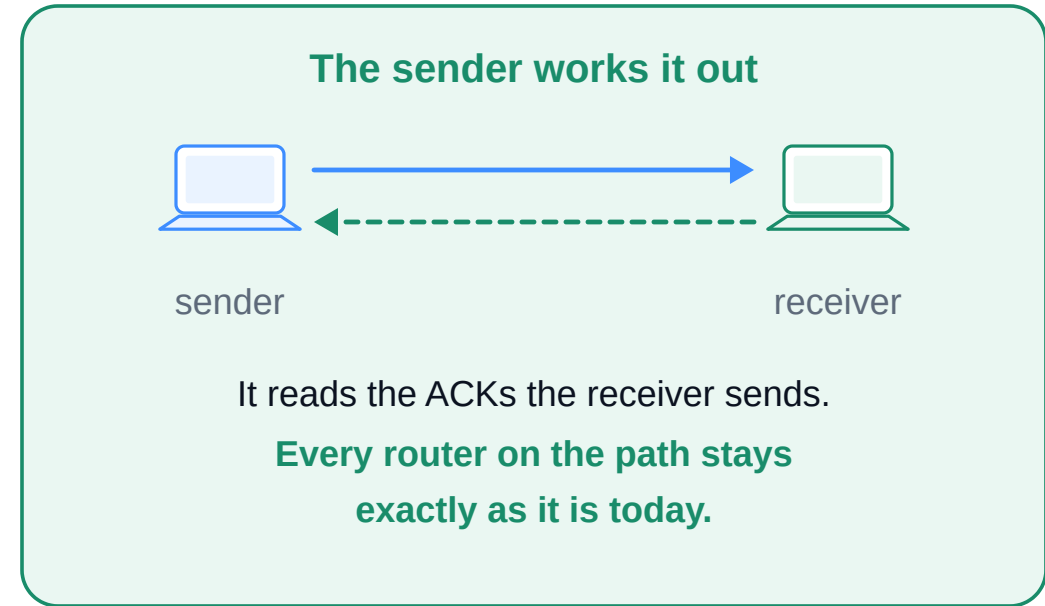
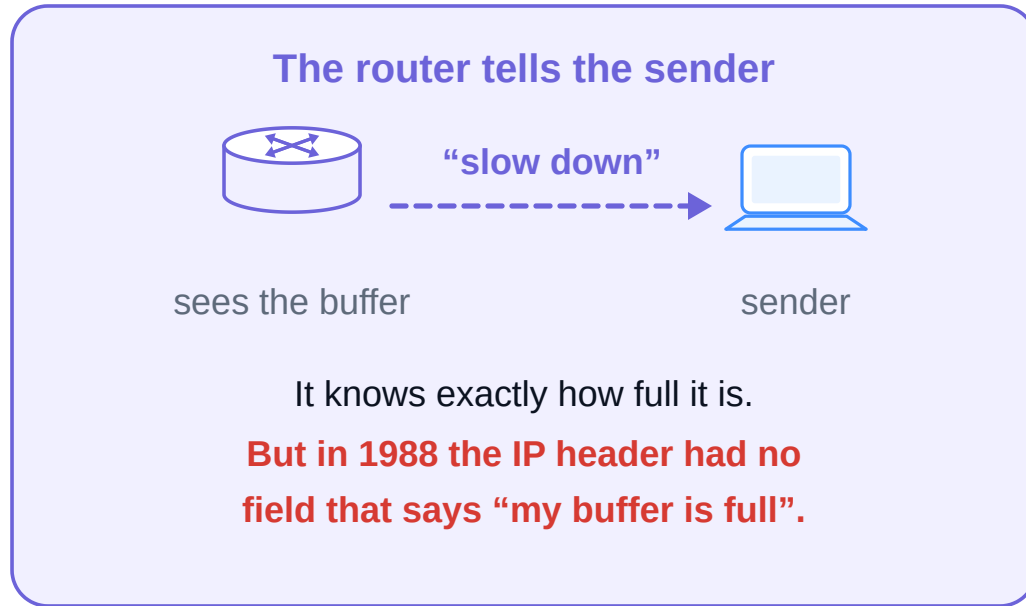
How can a sender find a rate that nothing tells it?

06

Probing for the rate

The sender can set `cwnd`, but it cannot see the bottleneck, and the network does not say how fast to go. So it finds out the only way it can: by trying a little faster, and reading what comes back.

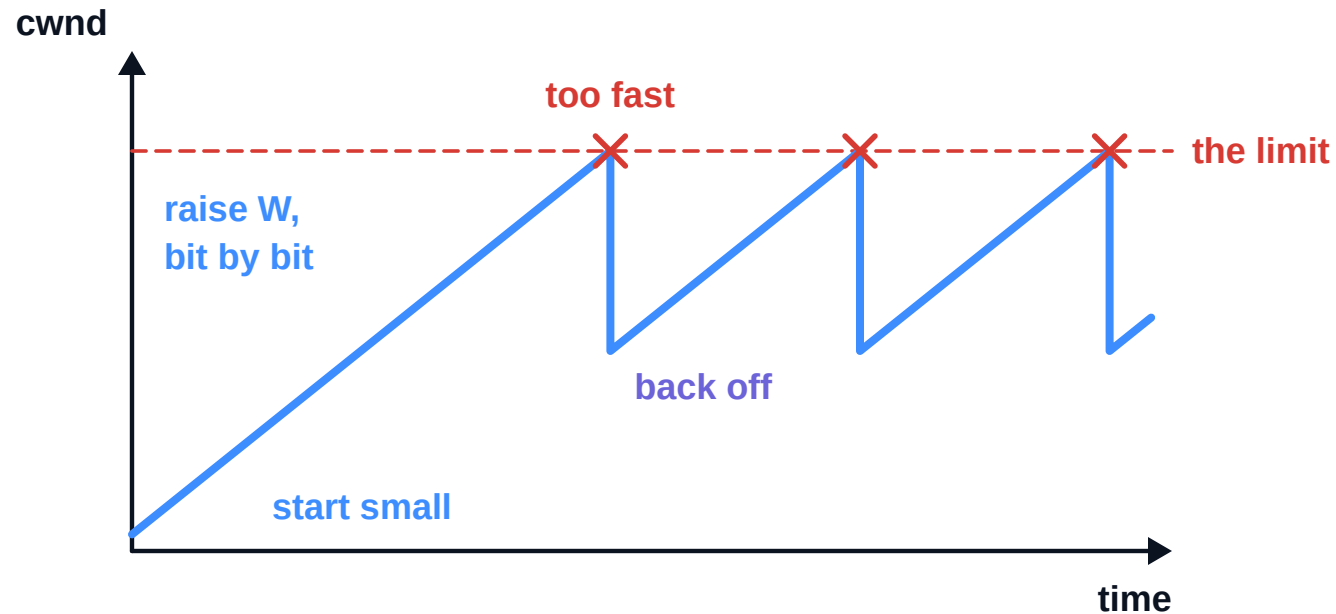
Will the network tell a sender how fast it may send?



The internet took the right-hand answer, and it is the only reason congestion control could be deployed at all.
The lecture after next comes back to the left-hand answer: ECN, the field added in 2001.

The network says nothing, so the sender and receiver work it out between them.

How does a sender find a limit it cannot see?



+ Start small, raise W

each step up sends a little faster than the one before

⚠ Until a signal: too fast

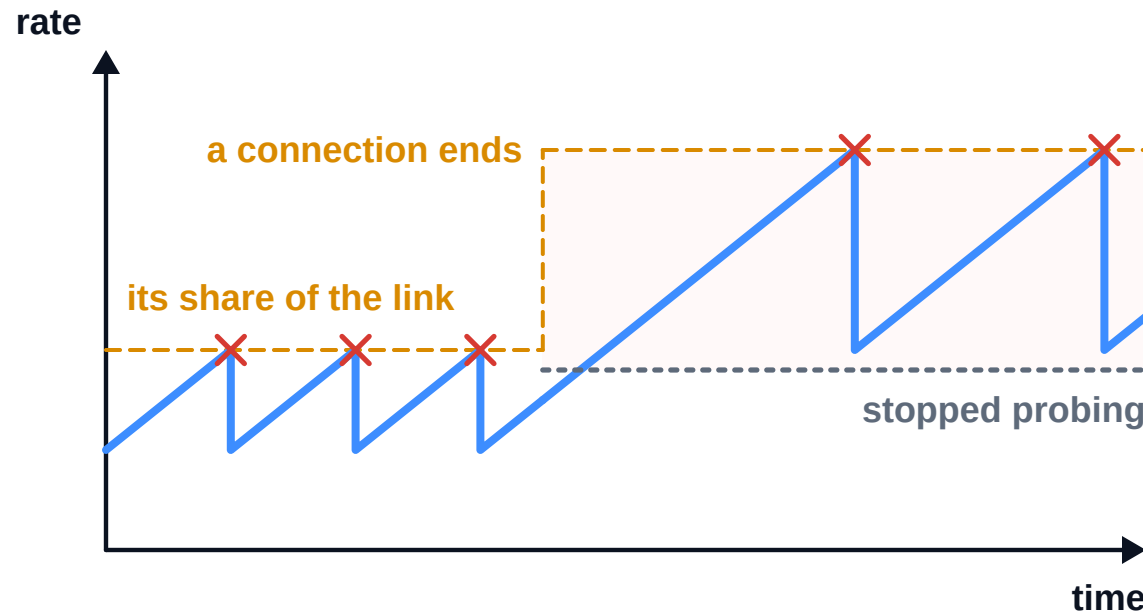
the sender cannot see the limit, but it can tell when it crossed it

🔄 Back off, then climb again

drop well below the limit, then raise W slowly once more

Climb until a signal says too fast, back off, and climb again.

Once it has found the limit, why keep probing?



✂ The limit moves

other connections start and end,
so this sender's share changes

⚠ A sender that stops probing

never finds the bandwidth
that someone else has freed

So the sawtooth never ends:
probing is the steady state.

Because the limit moves. Only a sender that keeps probing can follow it.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

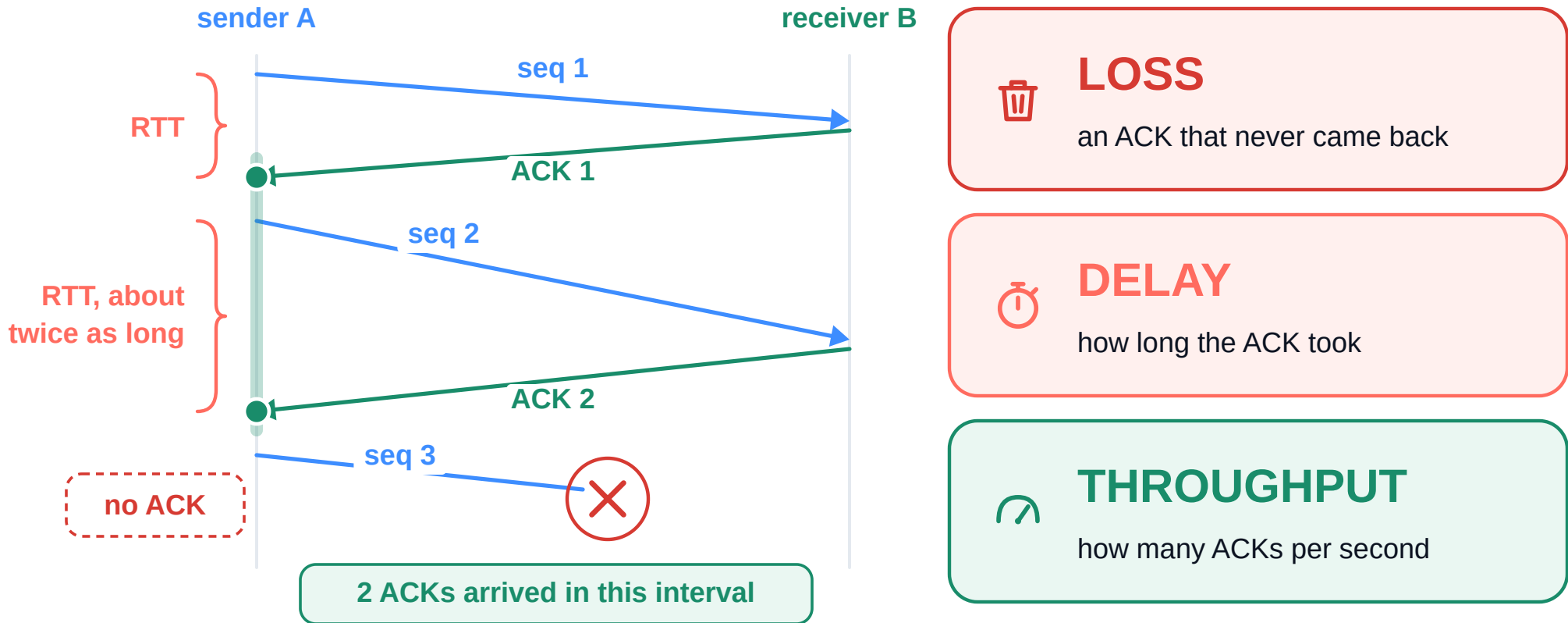
7 AIMD

8 Throughput

9 Slow start

10 On a loss

What can a sender measure from the ACKs that come back?



Loss, delay and throughput: three measurements, and no message from the network.

Which of the three should the sender use as its signal?



TCP uses loss.

A lost packet means a buffer on the path was full: the sender went too fast.

✓ It was already there

every sender had to detect loss anyway, in order to retransmit



It needs no new field

no header bit, no new message, no change to any router



And it is usually right

wired links rarely damage packets, so a loss is almost always a full buffer

Delay and throughput carry information too, and many newer algorithms use them.

This lecture builds the classic loss-based rule first; the next lecture comes back to them.

Loss is the too-fast signal: probing climbs until it sees one.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

8 Throughput

9 Slow start

10 On a loss

How should cwnd go up, and how should it come down?

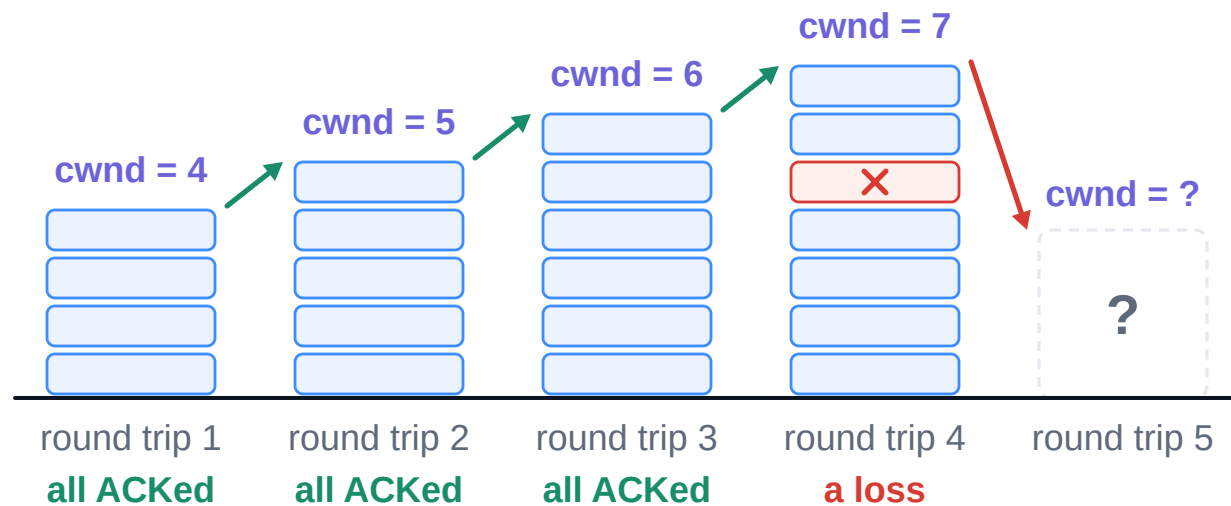
07

AIMD

Probing raises cwnd until a loss, then lowers it. This part pins down when each step happens and how big it can be: two ways up, two ways down, four rules. Alone on a link any of them works; when senders share a link, only one is fair.

What makes cwnd go up, and what makes it come down?

each column is one round trip; each box is one packet of its window



✓ Up: every packet ACKed

the path carried the whole window, so the next round trip sends more: one step up per round trip

⚠ Down: a packet lost

a buffer overflowed, so the next round trip sends less: one step down, however many were lost

But a sender sees ACKs one at a time, not round trips. How does it know when one has passed?

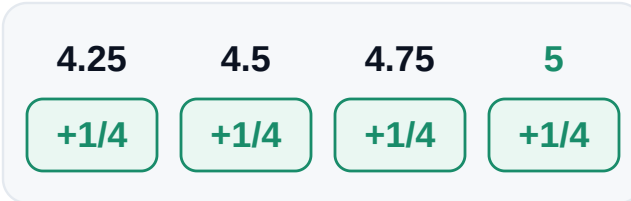
One decision per round trip: every packet ACKed, step up; a loss, step down.

How does a sender add one packet per round trip?

The sender has no clock that ticks once per round trip. What it has is ACKs, one for each packet that got through.

each ACK adds $1/\text{cwnd}$
+1 per round trip
additive

round trip 1: $\text{cwnd} = 4$, so 4 ACKs



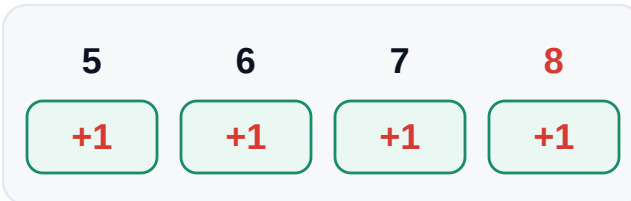
round trip 2: $\text{cwnd} = 5$, so 5 ACKs



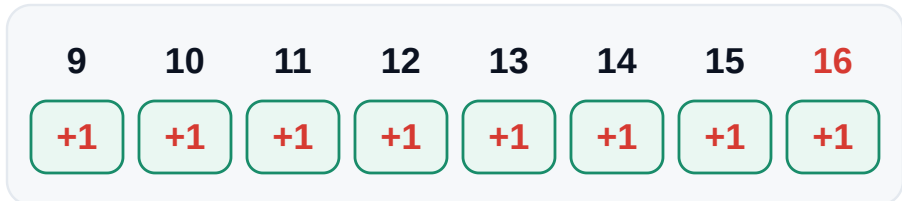
cwnd may be a fraction; the sender still sends whole packets, the fifth once cwnd reaches 5

each ACK adds 1
×2 per round trip
multiplicative

round trip 1: $\text{cwnd} = 4$, so 4 ACKs



round trip 2: $\text{cwnd} = 8$, so 8 ACKs



A round trip is cwnd ACKs, so the per-round-trip rule is carried out per ACK, with no clock.

Adding a whole packet per ACK is not additive at all: it doubles cwnd every round trip.

Each ACK adds $1/\text{cwnd}$, so one round trip of ACKs adds exactly one packet.

- 1 Congestion
- 2 Causes
- 3 Challenges
- 4 The costs
- 5 The rate
- 6 Probing
- 7 AIMD
- 8 Throughput
- 9 Slow start
- 10 On a loss

How big is a step? Two ways up, two ways down



No loss this round trip: step up

AI

additive increase

add a fixed amount: $\text{cwnd} + a$

$a = 1$ (+1/cwnd per ACK): 4, 5, 6, 7 ...

MI

multiplicative increase

multiply by $b > 1$: $\text{cwnd} \times b$

$b = 2$ (+1 per ACK): 4, 8, 16, 32 ...



A loss: step down

AD

additive decrease

subtract a fixed amount: $\text{cwnd} - a$

$a = 1$: 32, 31, 30 ...

MD

multiplicative decrease

multiply by $c < 1$: $\text{cwnd} \times c$

$c = \frac{1}{2}$: 32, 16, 8 ...

Additive moves by the same step however big cwnd is; multiplicative moves by the same fraction.

So a multiplicative step is small when cwnd is small, and violent when it is large.

One choice up, one choice down: four possible rules.

Additive is a fixed step, multiplicative a fixed fraction, up and down.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

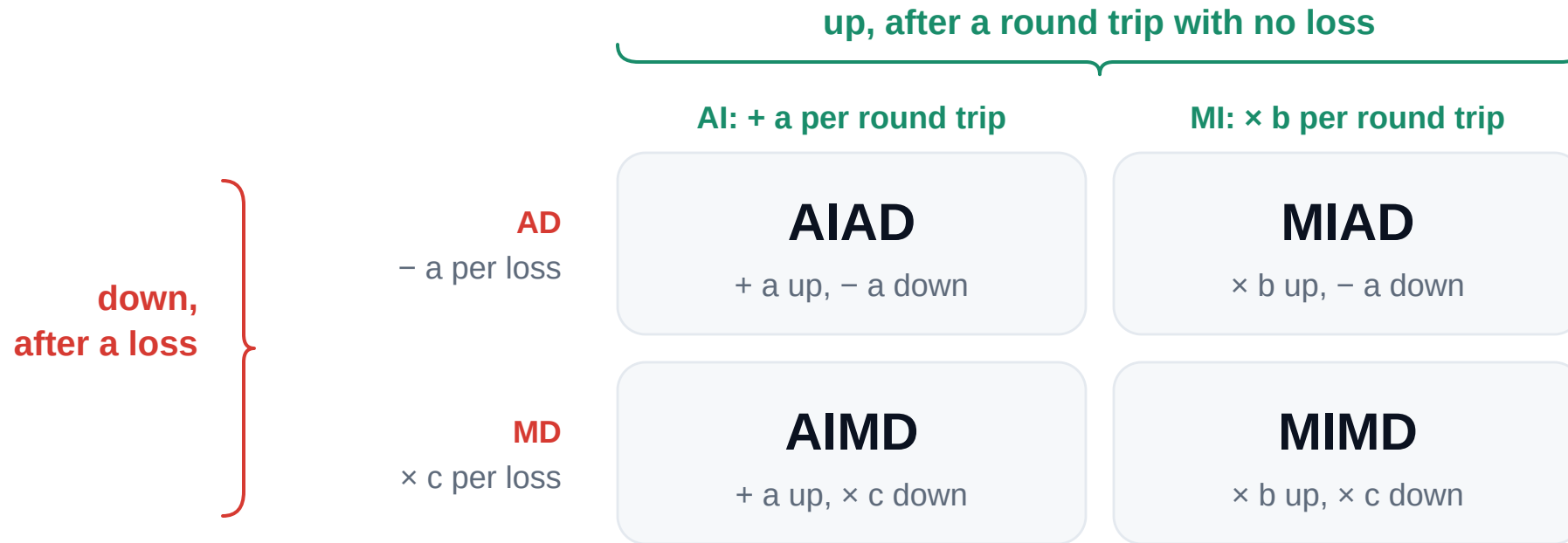
7 AIMD

8 Throughput

9 Slow start

10 On a loss

Four possible rules. Which one should a sender use?

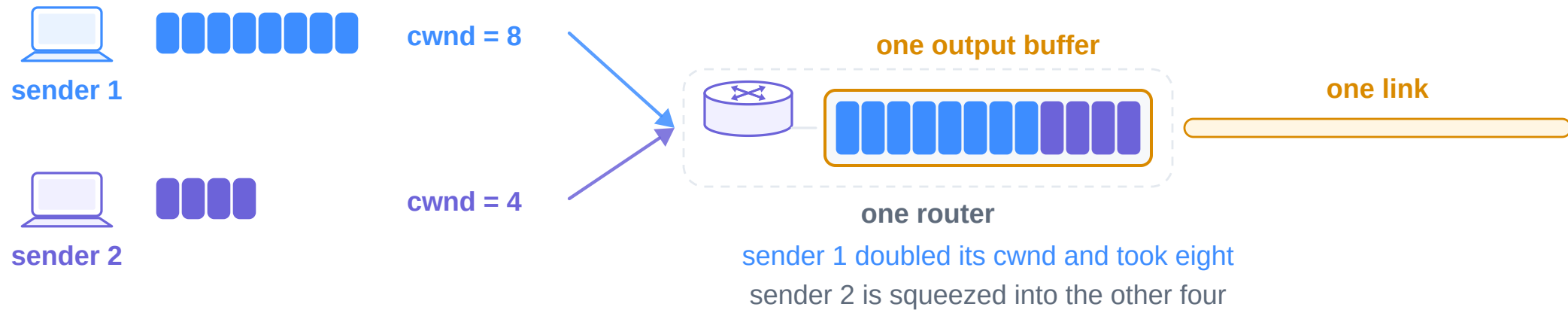


Alone on a link, all four work: each finds the limit and oscillates around it.
The choice matters when senders share a link, and that is the case to look at.

Alone, any of the four will do. What matters is what happens when senders share.

- 1 Congestion
- 2 Causes
- 3 Challenges
- 4 The costs
- 5 The rate
- 6 Probing
- 7 **AIMD**
- 8 Throughput
- 9 Slow start
- 10 On a loss

Two senders, one bottleneck: who gets how much?

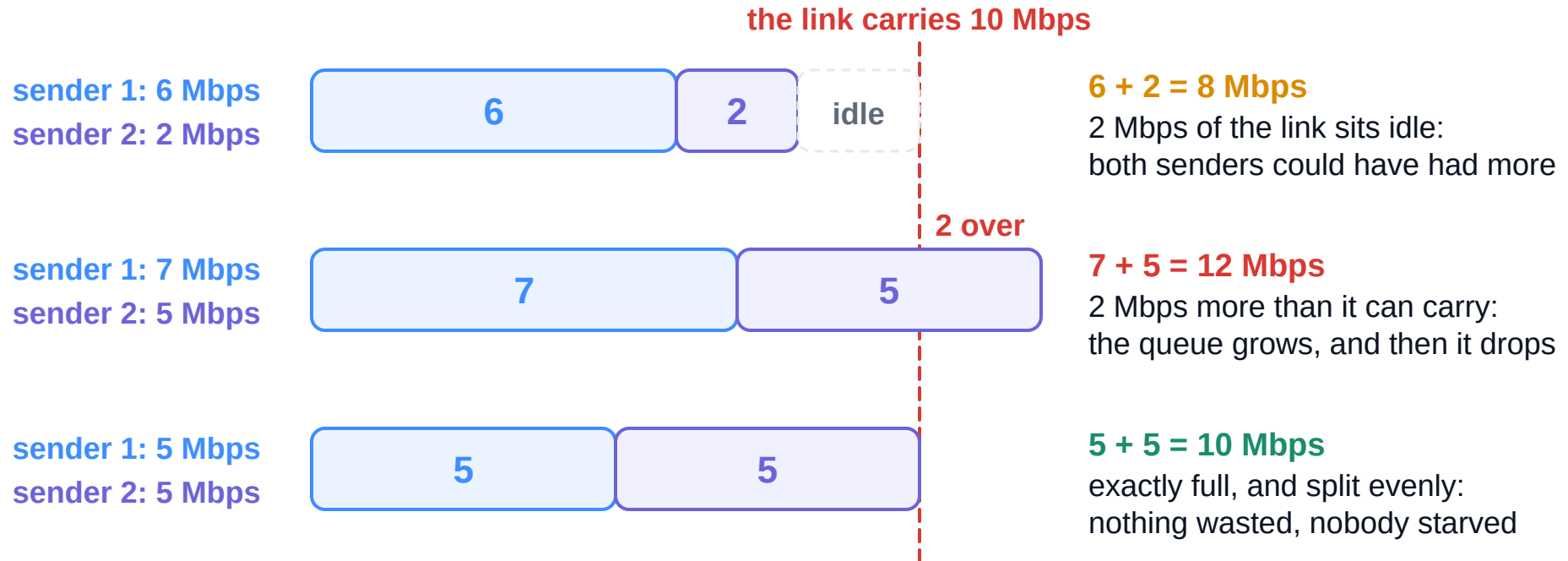


We want the two to get equal shares of the link.

But neither knows the other exists, and they cannot talk: the rule itself has to get them there.

A fair split has to come out of the rule itself: the two senders never coordinate.

Two senders on one 10 Mbps link: three ways it can go

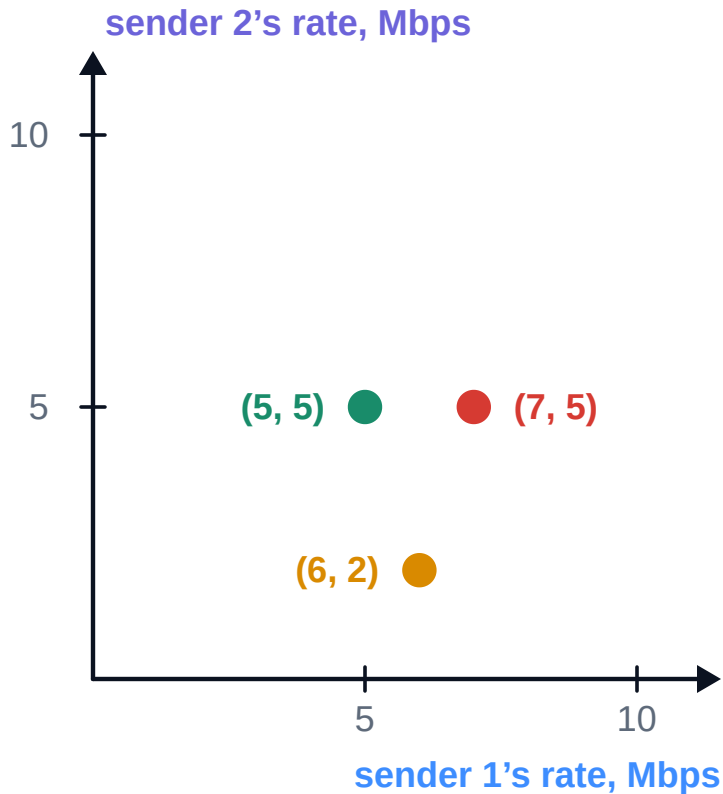


What each sender is doing right now is just a pair of numbers. Here are three of the pairs.

Too little wastes the link, too much fills the buffer, and one pair does neither.

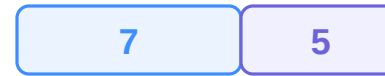
1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

Each of those pairs is one dot on a chart



(6, 2)

the link is not full



(7, 5)

more than the link can carry



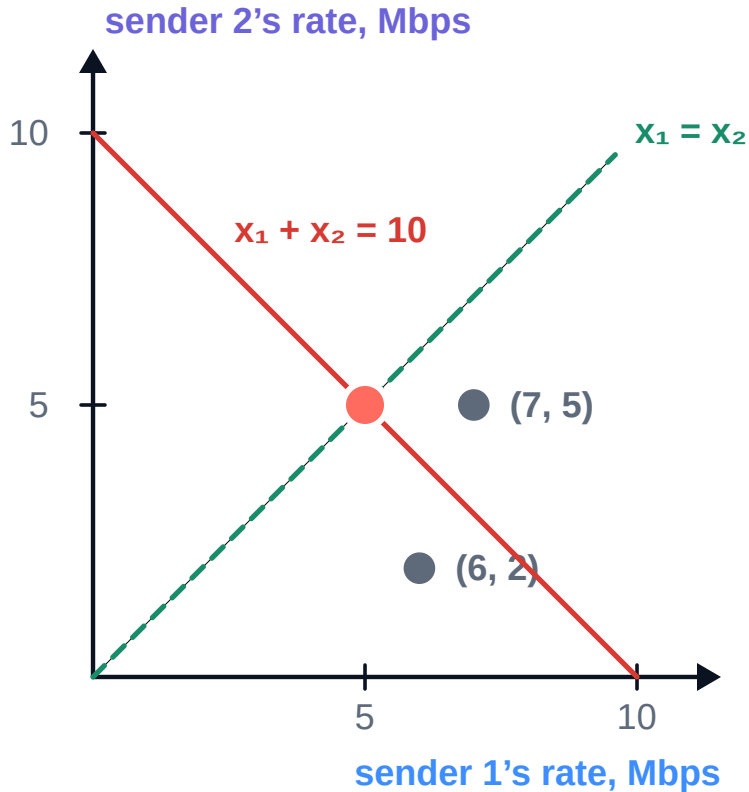
(5, 5)

full, and split evenly

Go right to sender 1's rate, up to sender 2's, and mark the spot.
No time axis here: one dot is one instant, and it moves
whenever one of the two rates changes.

The chart is not a graph over time. It is a map of who is sending how much.

Two senders, one link: every possible split



The efficiency line

$$x_1 + x_2 = 10$$

every pair that exactly fills the link
 (6, 2) sits below it: capacity wasted
 (7, 5) sits above it: the buffer grows

The fairness line

$$x_1 = x_2$$

every pair where the two get the same
 (7, 5) is off it: sender 1 is taking more
 a point on it may still waste the link

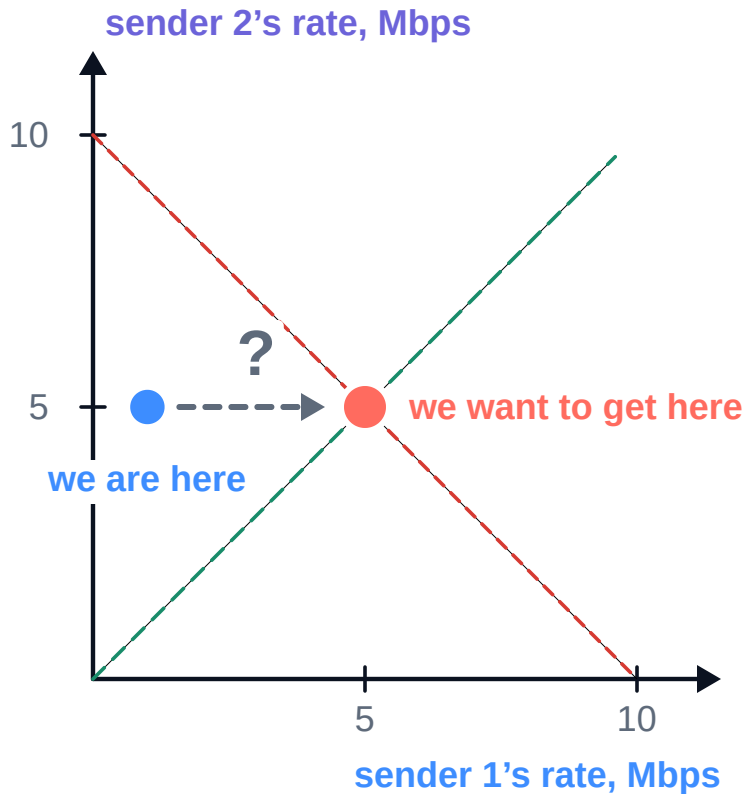
Where they cross

$$(5, 5)$$

full link and equal shares at once
 the only pair that is both
 a rule has to push any point here

Each point is one way to split the link. A rule is a way of moving that point.

How do the senders get the point to the middle?



The target

(5, 5): the link full and the two shares equal: the one point that is both

What neither sender can see

the other sender's rate, where the two lines are, or that (5, 5) exists at all

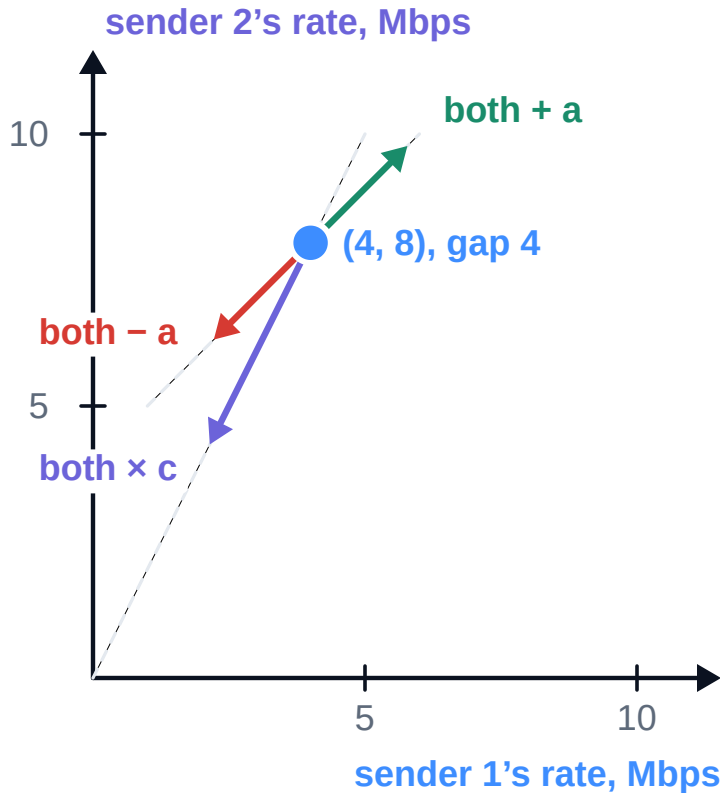
So what is a sender left with?

its own cwnd, and one bit coming back: a loss, or no loss. Both run the same rule.

Both raise cwnd with no loss, lower it on a loss.
Which of the four rules brings the point here?

Nobody can steer towards a point they cannot see. The rule has to get there blind.

Which way does the point move when cwnd changes?



Both senders add a the same step for each of them
the point slides up a 45° line, away from the origin
(4, 8) → (6, 10): the gap is still $10 - 6 = 4$

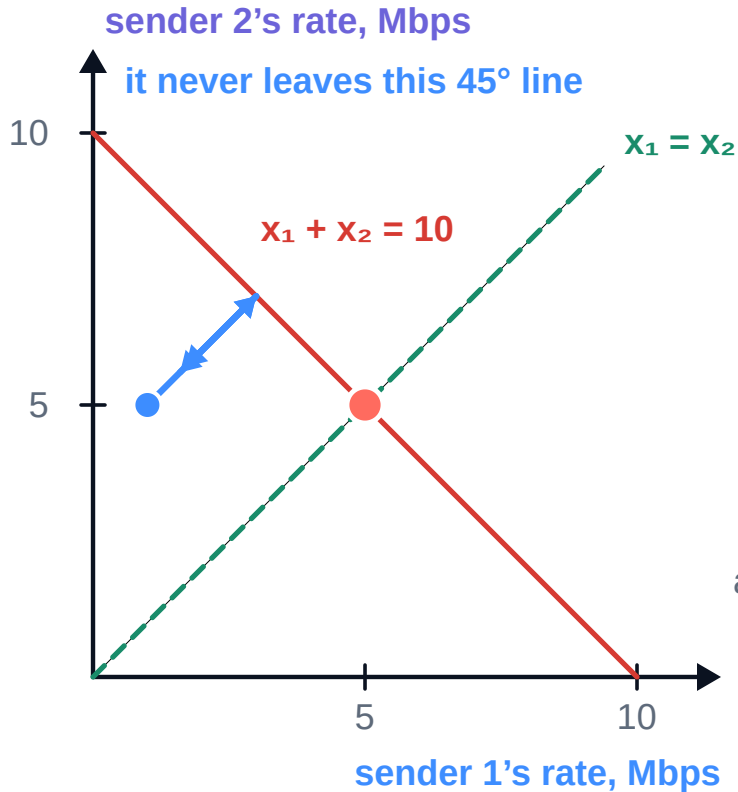
Both senders subtract a the same step back
down the very same 45° line, towards the origin
(4, 8) → (2, 6): the gap is still $6 - 2 = 4$

Both senders multiply by c the same fraction for each
down the line that joins the point to the origin
(4, 8) → (2, 4): the gap is $4 - 2 = 2$, halved

Up while no loss is heard, down on every loss,
and only the third move changes the gap.

Adding slides along 45°. Multiplying slides towards the origin, and shrinks the gap.

Add on the way up, subtract on the way down?



attempt 1 of 3
AIAD

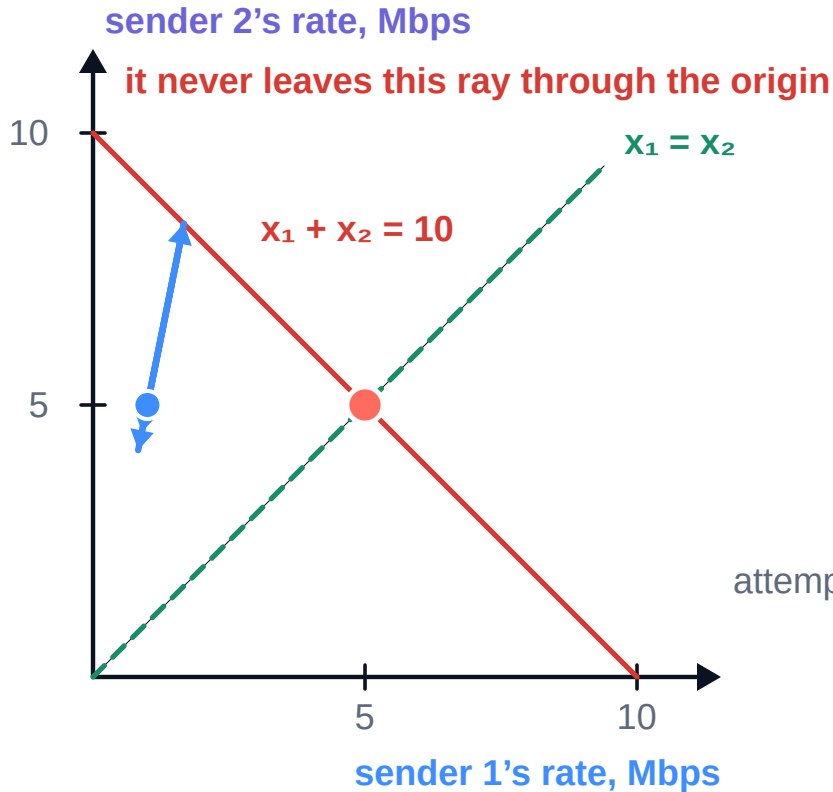
+ A round trip with no loss: both add a
the point moves up and to the right along a 45° line

↓ A loss: both subtract a
the point moves back down the same 45° line

The gap between the two senders never changes. AIAD is not fair.

The link fills, and whatever unfairness it started with, it keeps.

Multiply on the way up, halve on the way down?



attempt 2 of 3
MIMD

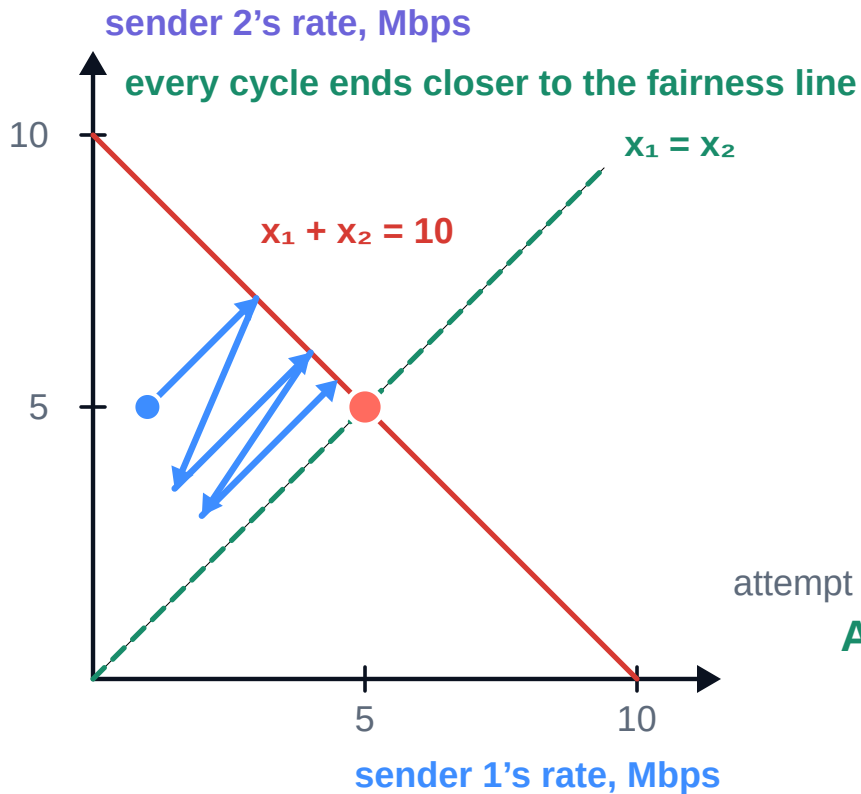
⚖️ Both moves are multiplications
a multiplication keeps the ratio
 $x_1 : x_2$ exactly as it was

🔄 So the split never moves
whatever ratio it started with,
it still has after a thousand cycles

MIAD is worse still: the gap grows
every cycle, so it diverges.

Multiplying keeps the ratio, so MIMD keeps whatever split it started with.

Add on the way up, halve on the way down?



+ A round trip with no loss: both add a the same 45° move: it keeps the gap between the two the same

↓ A loss: both multiply by c
the point moves towards the origin, which shrinks the gap between them

**Additive increase keeps the gap.
Multiplicative decrease shrinks it.**

AIMD converges to an equal share from any starting point. That is why TCP uses it.

Why does the gap shrink, cycle after cycle?

what happens	sender 1	sender 2	the gap
start	1	5	4
grow: + 2 each, until the link is full	3	7	4
loss: halve both	1.5	3.5	2
grow: + 2.5 each	4	6	2
loss: halve both	2	3	1
grow: + 2.5 each	4.5	5.5	1
loss: halve both	2.25	2.75	0.5
grow: + 2.5 each	4.75	5.25	0.5
... and so on	→ 5	→ 5	→ 0

Each grow row is many round trips of + a, until the link is full again:
from a total of 5, that is 2.5 each.

Growing cannot change the gap

it adds the same number to both,
so the gap comes through unharmed

Halving must change the gap

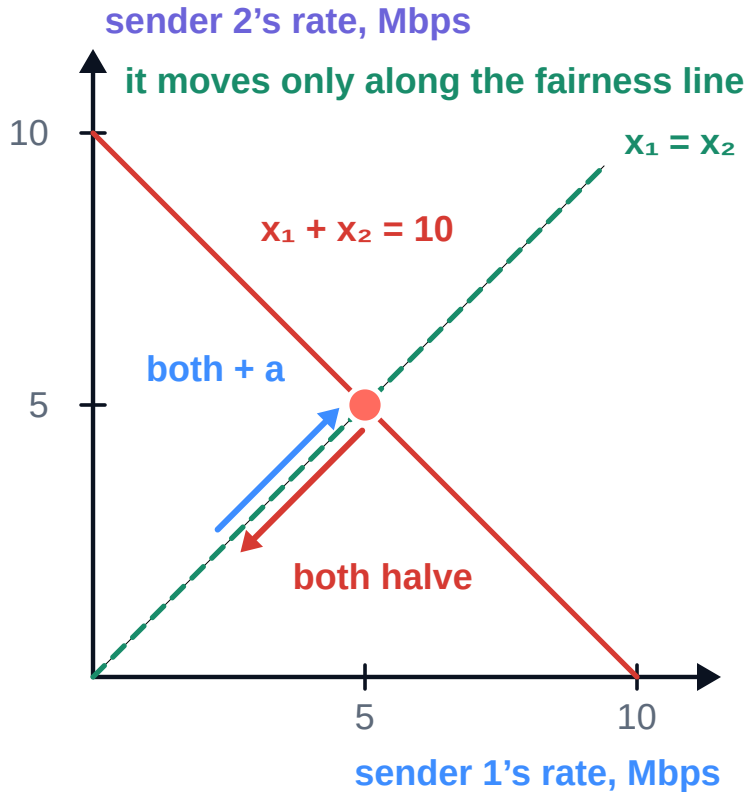
it multiplies both by one half,
so the gap is halved as well

So the gap halves every cycle

4, 2, 1, $\frac{1}{2}$, $\frac{1}{4}$... → 0, and the increase
keeps the point on the full link

The gap survives every increase and is halved by every decrease, so it goes to zero.

AIMD reaches the middle. How does it stay there?



✓ Equal stays equal

at $x_1 = x_2$ both senders make the same move, so a gap of 0 stays 0

🔄 But it never stands still

it cannot see the middle, so both keep adding until a loss, halve, and climb again

↔ Pushed off, it comes back

a loss that only one sender sees opens a gap, and every cycle after that halves it again

Equal stays equal; the point swings up to the middle and back, once per loss.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 **AIMD**

8 Throughput

9 Slow start

10 On a loss

AIMD is fair. How fast does an AIMD flow actually go?

08

AIMD's throughput

Part 7 found the rule and left its two steps as a and c . TCP fixes them: one packet per round trip up, half the window down. This part turns those two numbers into a rate you can compute.

Additive increase: one more packet per round trip

Part 7 drew rates. TCP writes the rule on cwnd.

$\text{rate} = \text{cwnd} \times \text{MTU} / \text{RTT}$: when both senders see the same RTT, a rule on cwnd is the same rule on the rate.

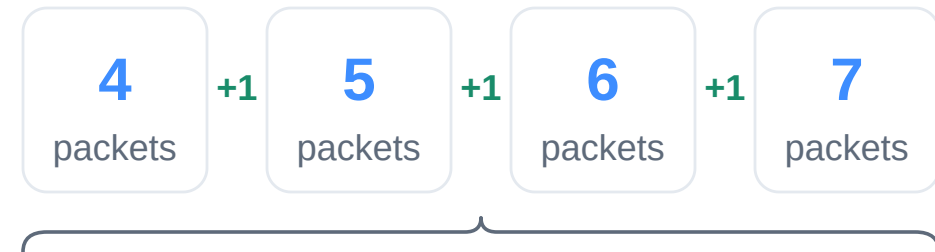


Additive increase

cwnd += 1 packet per round trip

in bytes that is $\text{cwnd} += \text{MTU}$, once every RTT,
so the window grows by 1,500 B per round trip

no matter whether cwnd is 4 packets or 4,000:
the step is the same size every time



four round trips

Per ACK, as in Part 7: $\text{cwnd} += 1/\text{cwnd}$ packet;
in bytes, $\text{cwnd} += \text{MTU} \times \text{MTU} / \text{cwnd}$.

The step never grows. A big window and a small one both gain one packet per round trip.

Multiplicative decrease: half the window on a loss

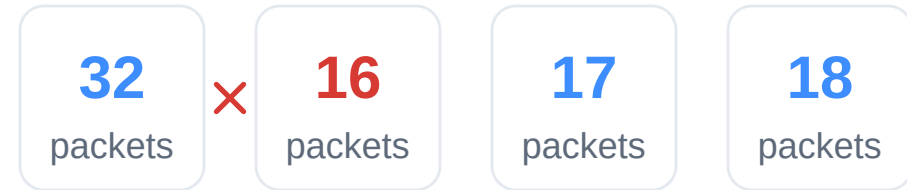


Multiplicative decrease

on a loss: $cwnd \rightarrow cwnd / 2$

the loss says the path could not hold $cwnd$ packets,
so the sender gives back half of what it had

in bytes: $cwnd = cwnd / 2$, and never
below two packets (RFC 5681)



a loss: halve it then climb again, +1 at a time

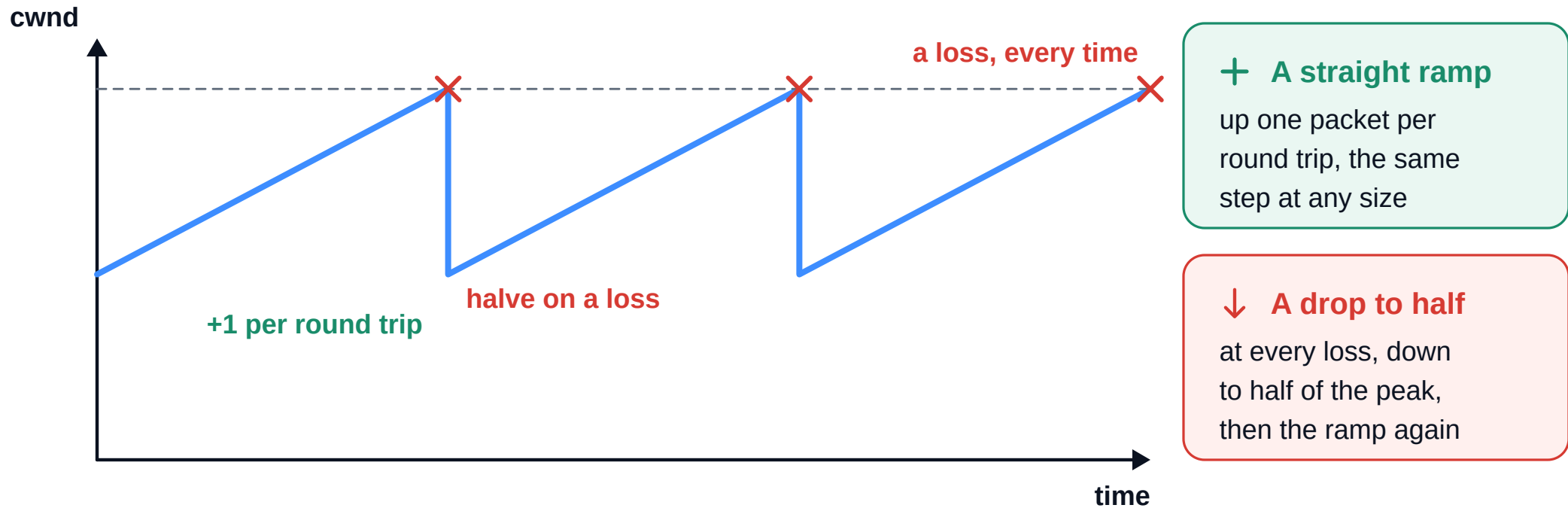
Halving is a fixed fraction, so the bigger the window was, the more it gives back, which is what clears a congested buffer quickly.

One packet more per round trip. Half the window on a loss.

Slow to take, quick to give back. The halving is also what made AIMD fair in Part 7.

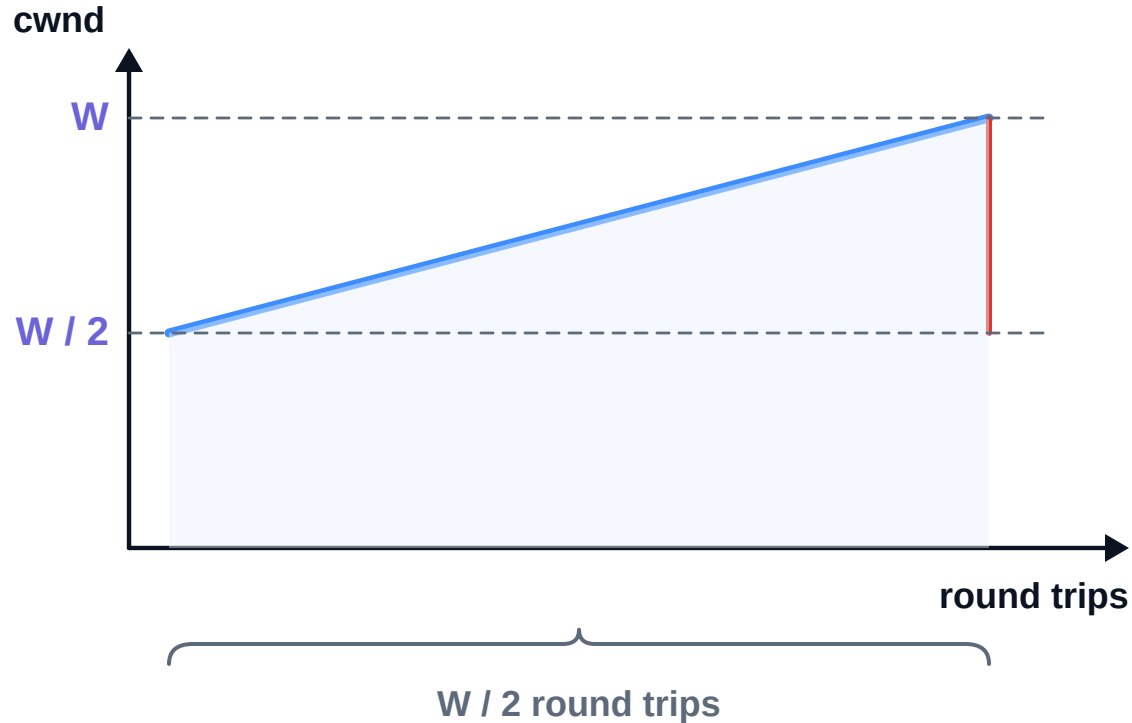
Give back a fraction, not a step: the more a sender had, the more it hands over.

What does cwnd look like under the two rules?



A sawtooth from half the peak up to the peak. Its average is the rate we want.

What is the average window over one sawtooth?



One tooth, start to finish

it begins at $W/2$, just after a loss halved it,
and climbs by one packet per round trip
so it takes $W/2$ round trips to get back to W

average window over the tooth

$$(W + W/2) / 2 = 0.75 W$$

the mean of a straight ramp: the shaded
area divided by how long it took

A connection gets three quarters of the largest window it managed to reach.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

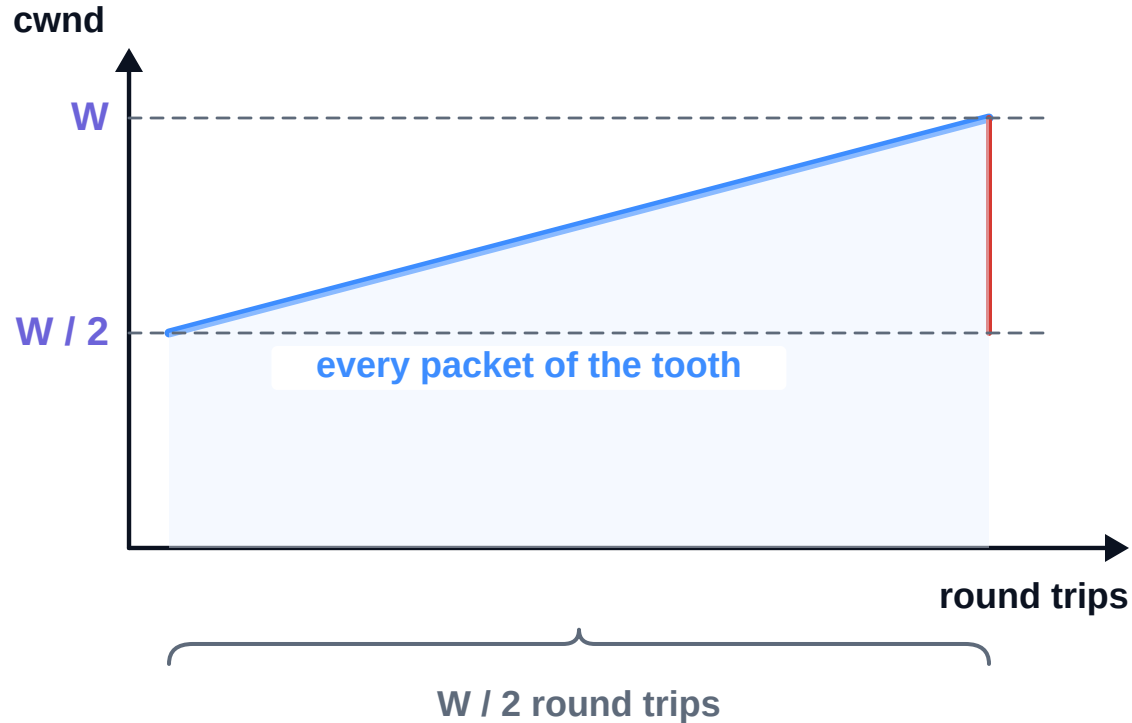
7 AIMD

8 Throughput

9 Slow start

10 On a loss

How many packets fit in one tooth?



packets sent in one tooth

$$\begin{array}{ccc} 0.75 W & \times & W / 2 \\ \text{per round trip} & & \text{round trips} \end{array}$$

$$= (3/8) W^2$$

and exactly one is lost

one drop ends every tooth, by definition of what a tooth is, so the loss rate is

$$p = 1 / (3/8) W^2 = 8 / (3 W^2)$$

One loss per tooth is the definition of a tooth, and that is where p comes from.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

7 AIMD

8 Throughput

9 Slow start

10 On a loss

From one loss per tooth to a throughput formula

we just counted

$$p = 8 / (3W^2)$$

one loss per tooth, and $(3/8)W^2$ packets in it

so turn it around

$$W = \sqrt{8 / 3p}$$

the window a sender reaches, given the loss rate

and the rate is $0.75 W$

$$\text{rate} = 0.75 W \cdot \text{MTU} / \text{RTT}$$

the average window, sent once per round trip

$$\text{put the three together: throughput} = 1.22 \cdot \text{MTU} / (\text{RTT} \cdot \sqrt{p})$$

$$0.75 \cdot \sqrt{8/3} = 1.22$$

Three substitutions, no new physics: count the tooth, invert it, average it.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

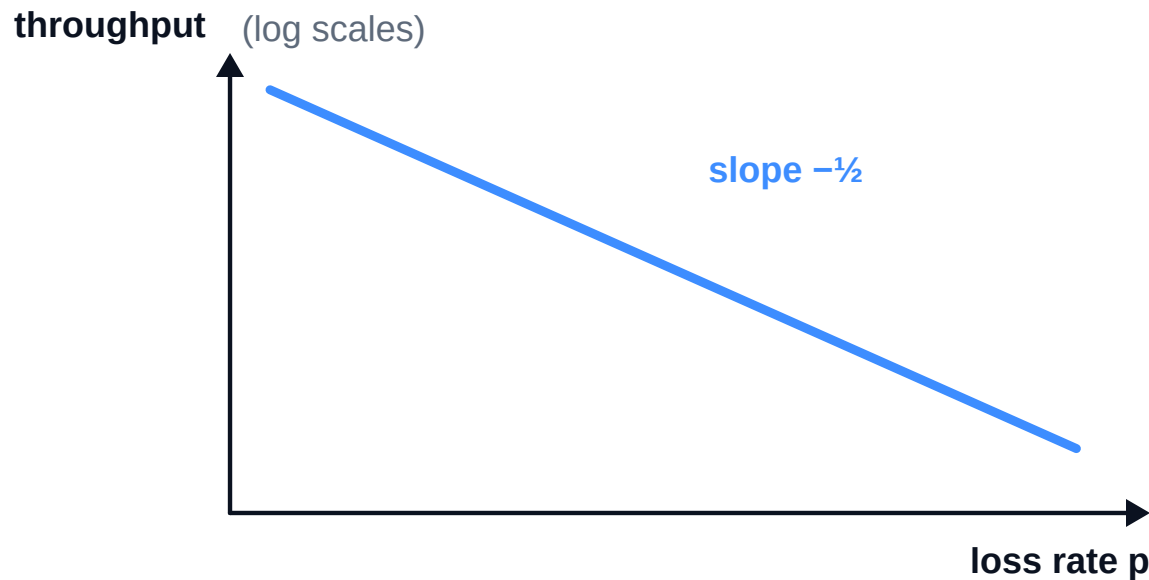
7 AIMD

8 Throughput

9 Slow start

10 On a loss

What the throughput formula says



The literature writes this with MSS, the TCP payload; here one packet is one MTU.

What is in the formula

RTT: double it and the throughput halves
 p : quadruple it and the throughput halves

a sender controls neither of them

What is not in the formula the link rate.

A 10 Gbps link and a 100 Mbps link give the same answer for the same RTT and loss rate, as long as the link is faster than the answer.

Throughput falls with the RTT and with the square root of the loss rate.

Checkpoint: MTU = 1,500 B, RTT = 100 ms, $p = 10^{-5}$

$$\text{throughput} = 1.22 \cdot \text{MTU} / (\text{RTT} \cdot \sqrt{p})$$

MTU
1,500 bytes

RTT
100 ms

p
 10^{-5}

throughput
46 Mbps

Now make the path worse: $p = 10^{-2}$ instead of 10^{-5} .

1.5 Mbps: a thousand times the loss rate costs about thirty times the throughput.

46 Mbps, even on a 10 Gbps link: here the loss rate, not the link, sets the rate.

Starting at $cwnd = 1$: how long to fill the path?

09

Slow start

AIMD adds one packet per round trip. The original TCP started at one packet, and a fast, long path holds thousands. How long does the climb take?

How many packets does a 1 Gbps, 30 ms path hold?

1,500 bytes

one packet, the MTU

1 Gbps

the bottleneck link

30 ms

the round-trip time

cwnd = 2,500

fills the path exactly

cwnd = 2,500

$2,500 \times 1,500 \text{ B} / 30 \text{ ms} = 1 \text{ Gbps}$

the window that keeps the link busy

cwnd = 1

$1 \times 1,500 \text{ B} / 30 \text{ ms} = 0.4 \text{ Mbps}$

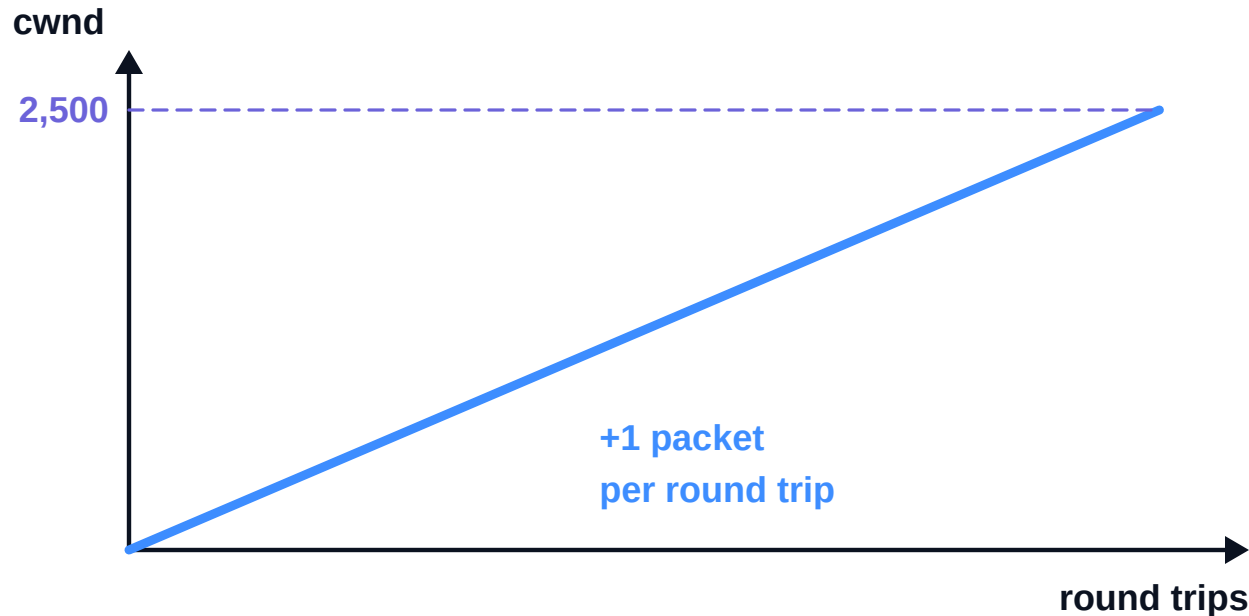
where the original TCP started

Every number in this part comes back to these four.

2,500 packets. A connection that starts at one packet has a long way to climb.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

How long does additive increase take to reach 2,500?



2,500 round trips

× 30 ms

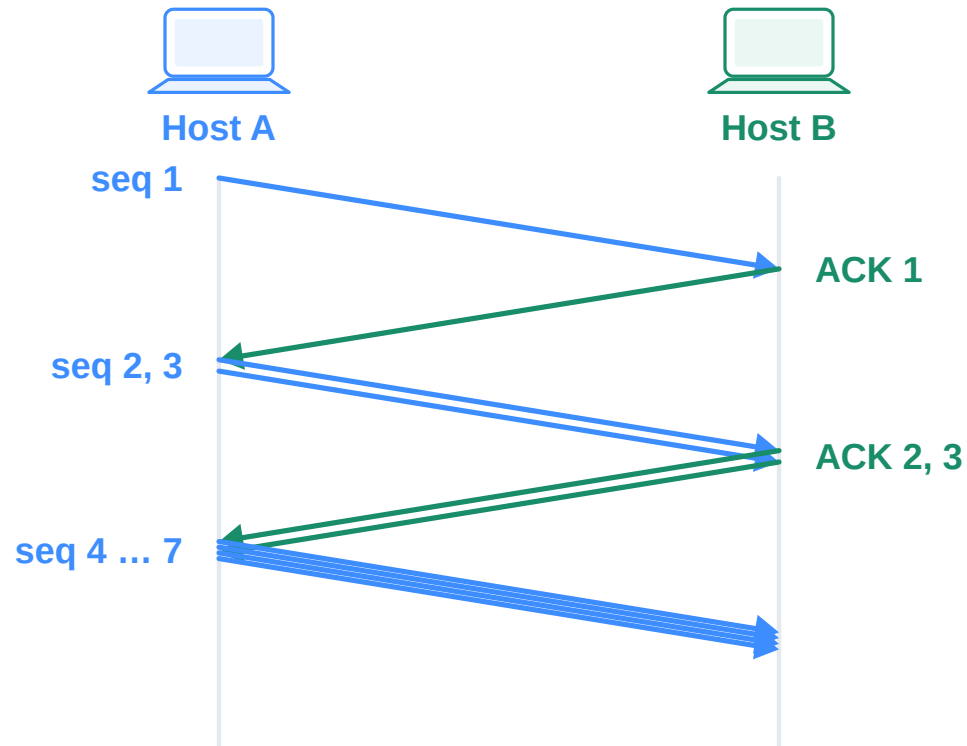
= 75 seconds

A web page is finished
in well under a second.

Most connections would
never use the link at all.

75 seconds. AIMD is the right rule for steady state and useless for starting.

What if every ACK released two packets?



The rule: one ACK, two packets

an arriving ACK frees the slot it is acknowledging, and the sender puts two packets into it, not one

cwnd += 1 packet per ACK

so a whole window of ACKs
doubles cwnd

1, 2, 4, 8, 16, ...

one doubling per round trip

Additive increase adds one packet per window. Slow start adds one per ACK.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

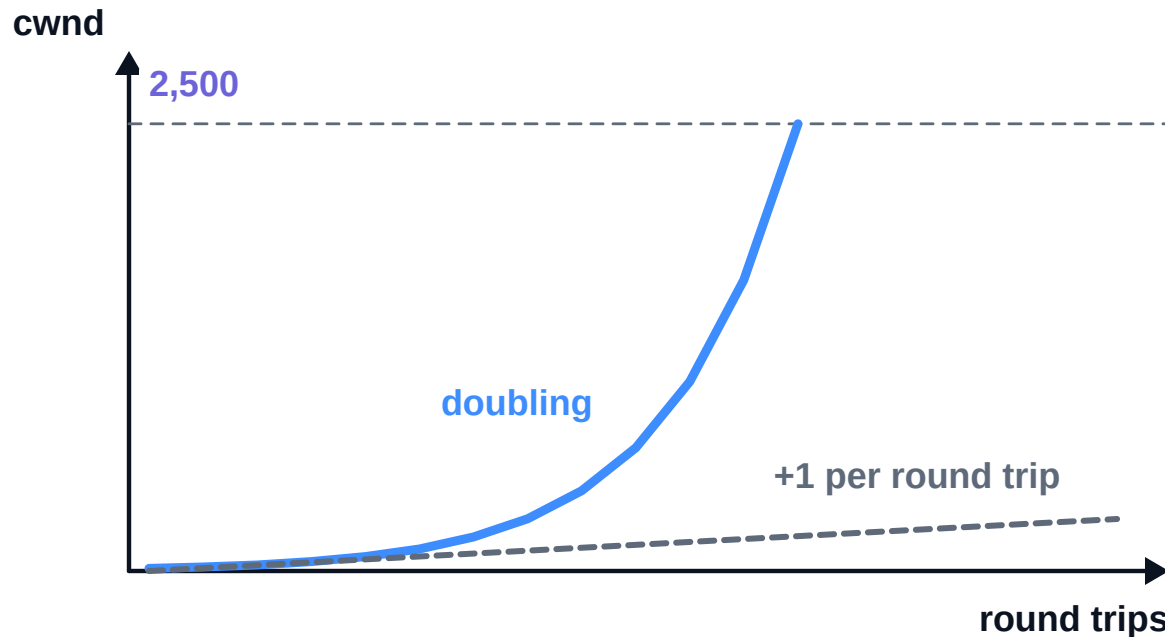
7 AIMD

8 Throughput

9 Slow start

10 On a loss

Doubling: twelve round trips instead of 2,500



additive, +1 each time

2,500 round trips

75 seconds

doubling

12 round trips

360 ms

twelve, because $2^{12} = 4,096$

the window passes 2,500 on the twelfth doubling, and $12 \times 30 \text{ ms}$ is 360 ms, well inside the life of a single web request

Doubling turns a 75-second climb into twelve round trips.

Why is doubling called “slow” start?

Before 1988



a full window of packets

on the link at once,
as soon as the connection opened

After 1988



one packet,

then two, then four:
the link never gets a full window at once

Slow start is slow compared with what it replaced,

not compared with additive increase, which it beats by a factor of two hundred.

The name is historical: it is slower than the burst it was invented to stop.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

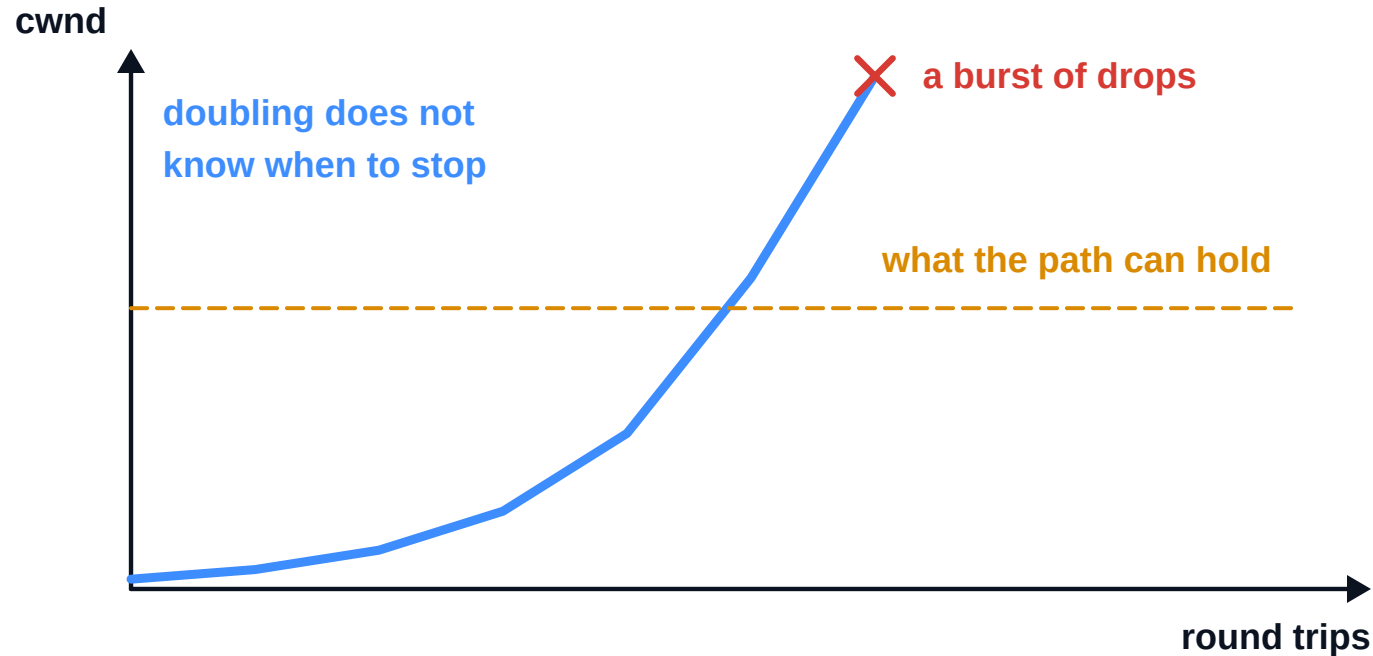
7 AIMD

8 Throughput

9 Slow start

10 On a loss

When should a sender stop doubling?



The last doubling

overshoots by as much as everything before it together.

So the sender needs a number that says
where to stop.

Before it overshoots, and nothing in doubling itself says where that is.

1 Congestion

2 Causes

3 Challenges

4 The costs

5 The rate

6 Probing

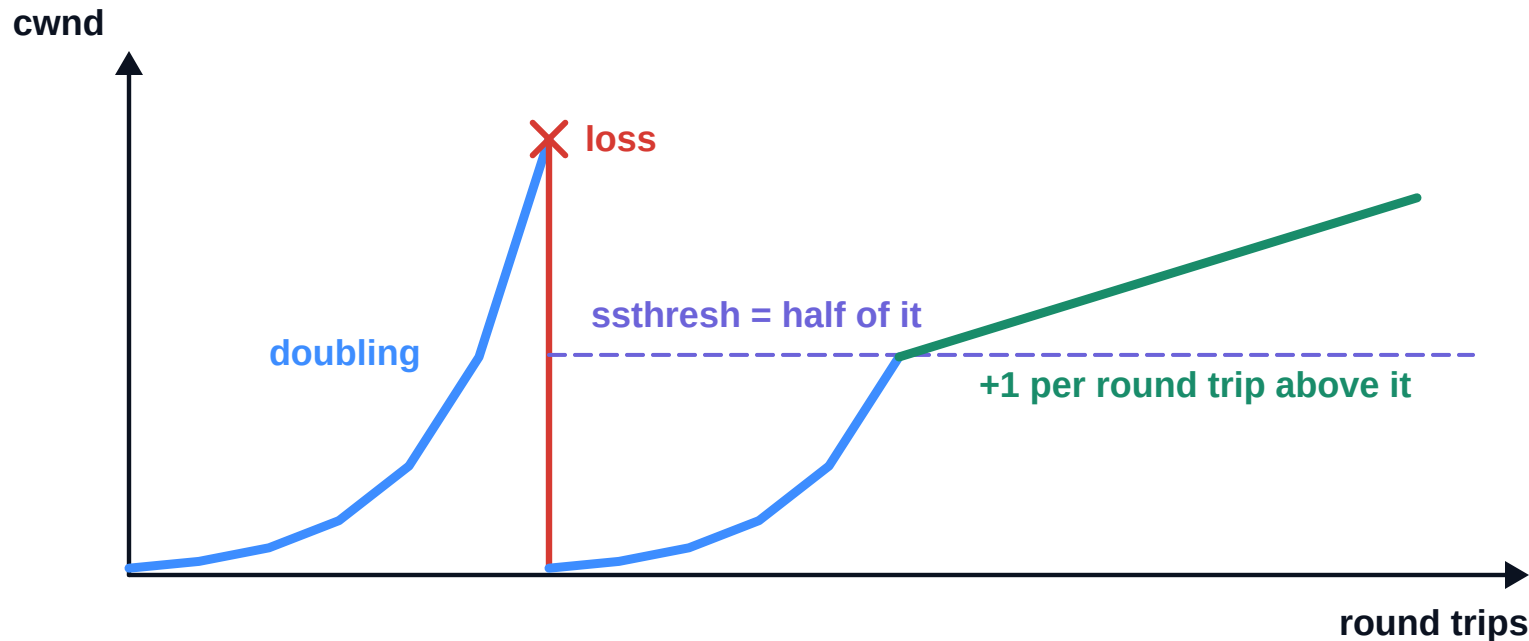
7 AIMD

8 Throughput

9 Slow start

10 On a loss

ssthresh: where doubling stops next time



The first slow start has no ssthresh yet, so it runs into a loss;
after a timeout cwnd goes back to 1 and doubles only up to ssthresh.

below ssthresh

double every
round trip

above ssthresh

add one per
round trip

on a loss

$\text{ssthresh} = \text{cwnd} / 2$

Half the window that overflowed: the sender's memory of how big the path is.

Why does a connection now start at ten packets?

IW = 10 packets, and at most 14,600 bytes

RFC 6928 writes it $IW = \min(10 \cdot MSS, \max(2 \cdot MSS, 14600))$, in TCP payload bytes.

A new connection may put ten packets on the link before it hears anything back.

cwnd = 1

12 round trips

360 ms

cwnd = 10

8 round trips

240 ms

Four round trips saved here ($\log_2 10 \approx 3.3$ in general), and a web page is a few round trips long.

Starting at one packet costs a short connection most of its lifetime.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

Are all losses equally bad news?

10

Reacting to loss

AIMD halves the window on a loss. But a sender learns about a loss in two different ways, and the two carry different amounts of evidence about whether the path still works at all.

Two ways a sender hears about a loss

Three duplicate ACKs



ACK 1, ACK 1, ACK 1

the receiver is answering
three separate later packets

A timeout



(silence)

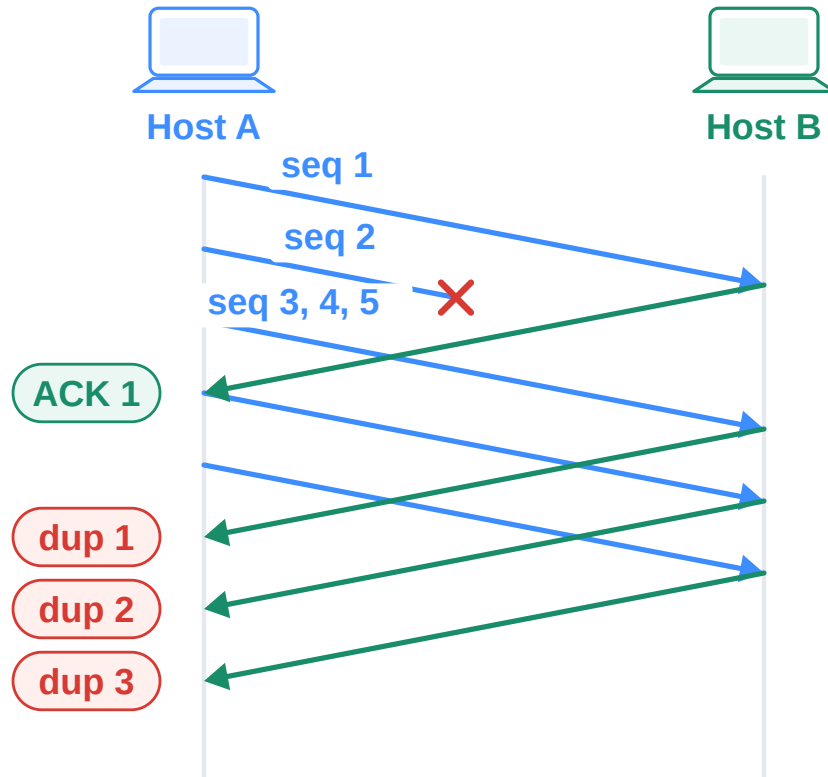
no ACK of any kind came
back for a whole timeout

❓ Same event, two reports. Do they deserve the same reaction?

They report the same lost packet and say very different things about the path.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

Three duplicate ACKs: packets are still getting through



✓ The path still works

seq 3, 4 and 5 arrived, so the links and the buffers are passing data

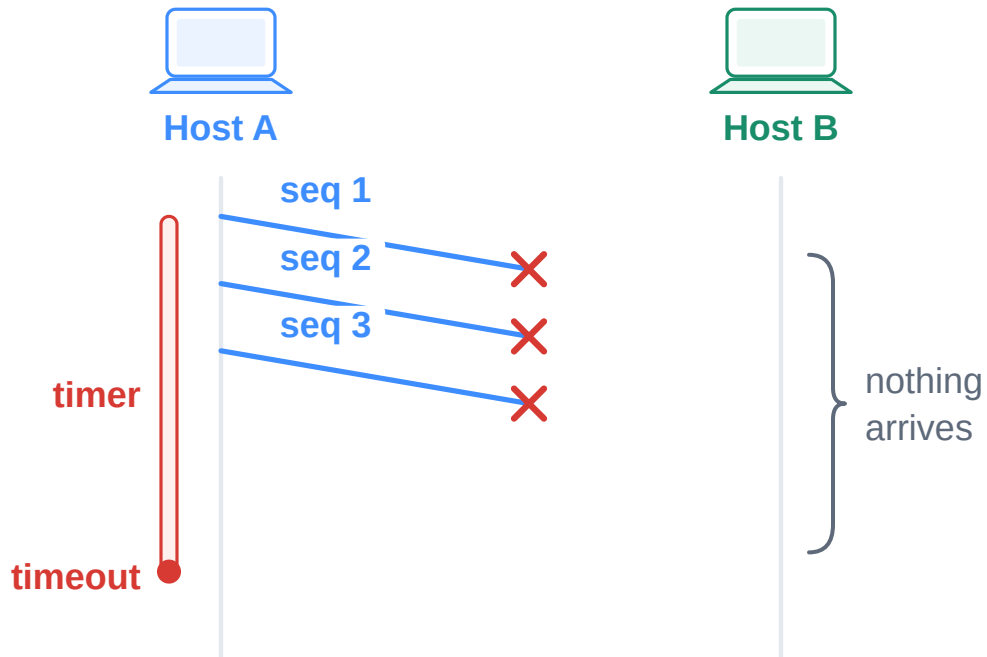
☑ The receiver is still there

it answered three more times: data gets through in both directions

One packet was dropped, but the buffer is not empty of this sender's packets, so it is still competing.

Evidence that the path carries traffic. Only one packet actually went missing.

A timeout: no sign that anything gets through



? No evidence of anything

not one ACK came back, so the sender has learned nothing at all

⚠ It could be much worse

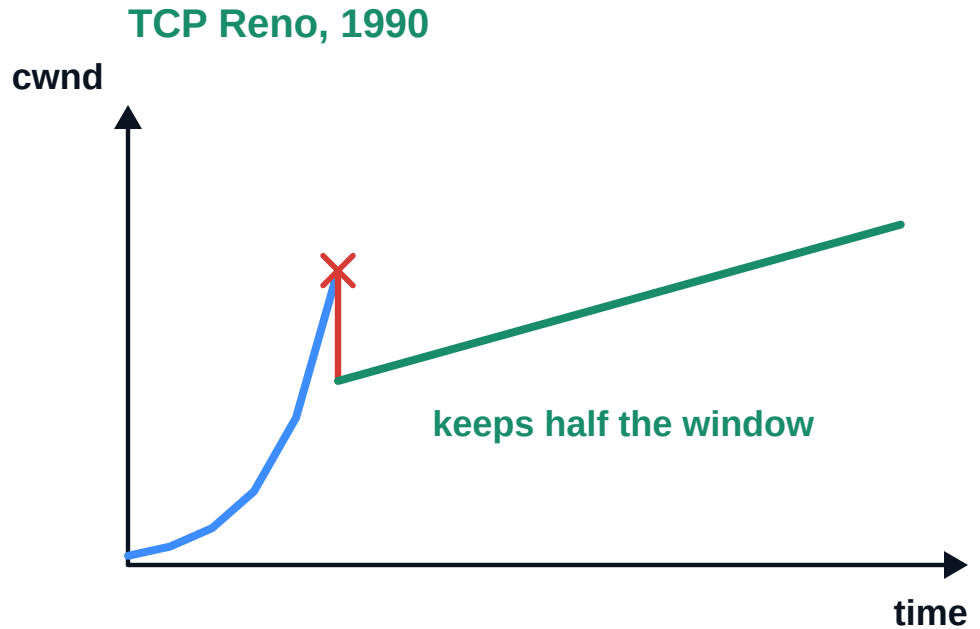
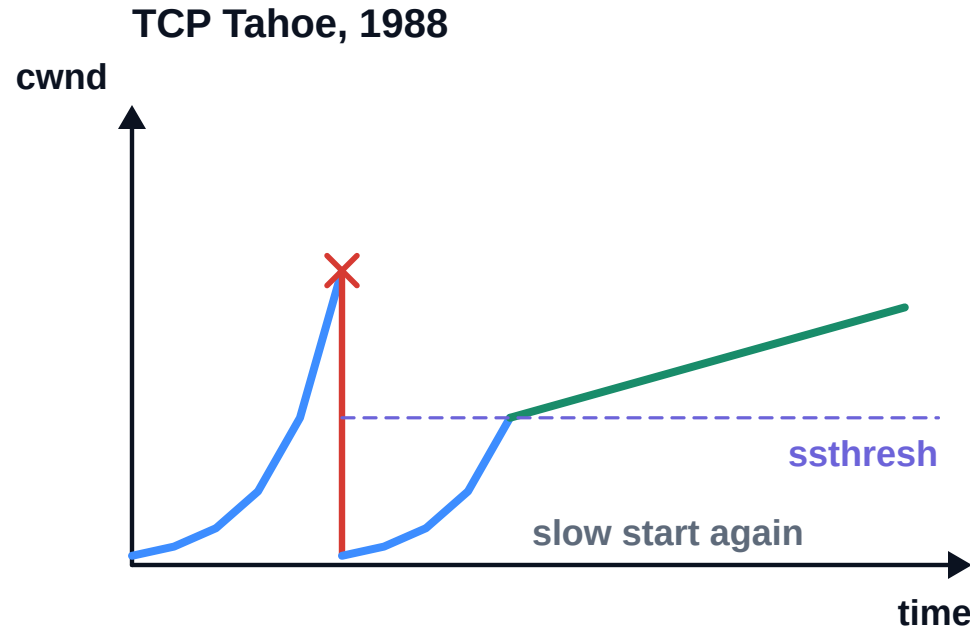
a route change, a buffer that is full of everybody, a dead link

With no evidence, the safe assumption is the worst one: start again from the beginning.

A window too small to bring back three duplicates also ends in a timeout, even on a path that works.

A timeout says the path may not work at all, so the sender restarts from $cwnd = 1$.

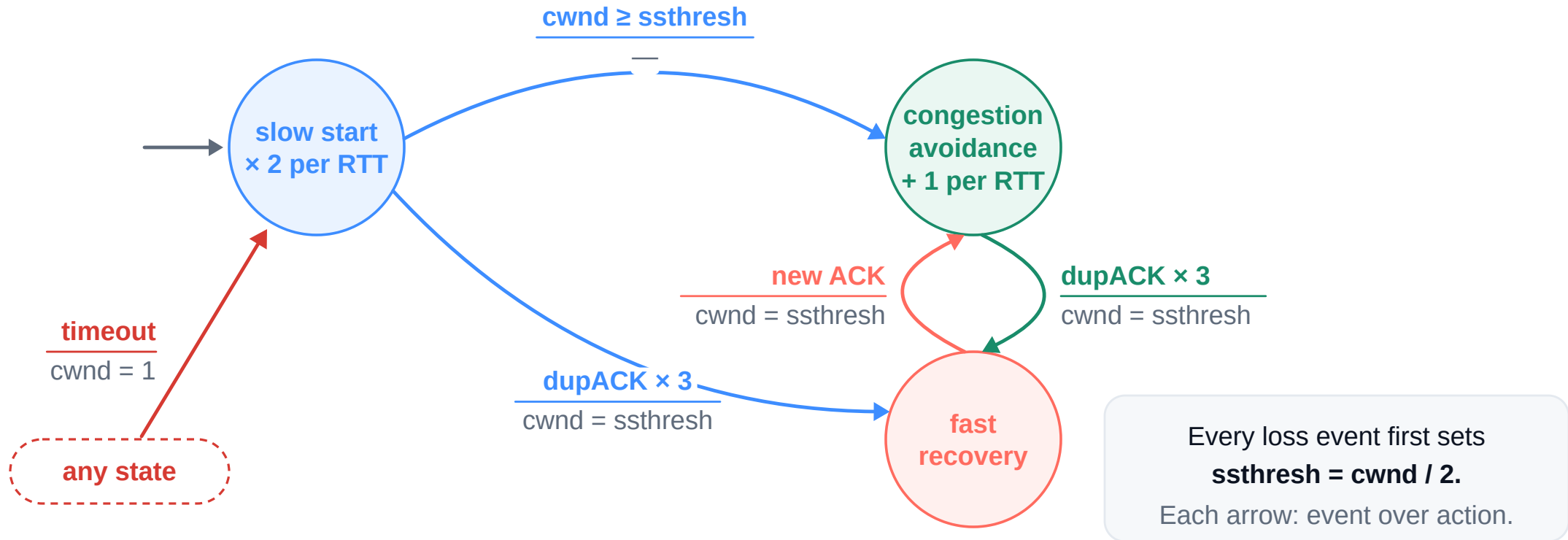
Tahoe and Reno react differently to the same two events



A timeout still sets cwnd = 1 in both. Only the duplicate-ACK case is treated differently.

Reno keeps half the window when it has evidence the path still works.

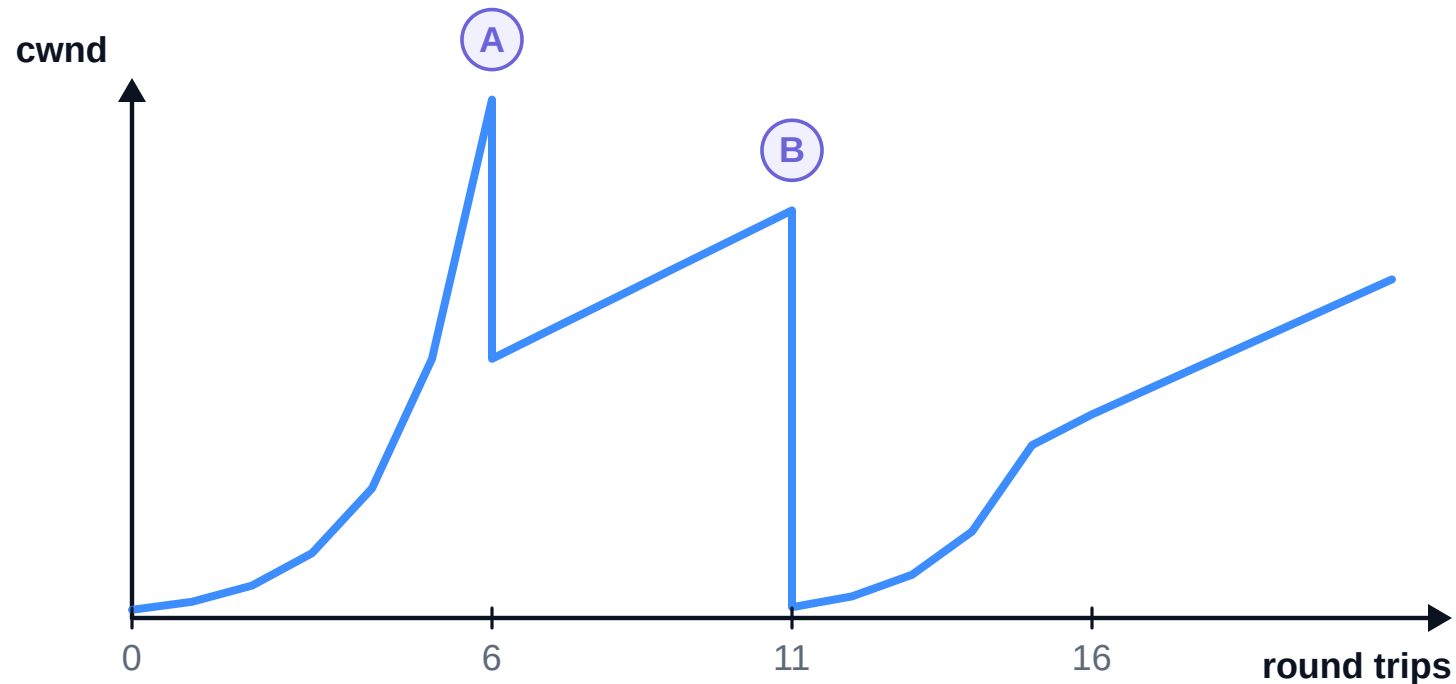
The whole algorithm as one state machine



Three states and four events: what every TCP sender ran through the 1990s.

- 1 Congestion
- 2 Causes
- 3 Challenges
- 4 The costs
- 5 The rate
- 6 Probing
- 7 AIMD
- 8 Throughput
- 9 Slow start
- 10 On a loss

Checkpoint: read the cwnd graph



Where is slow start?
the doubling stretches:
0–6 and 11–16

Which loss was A?
three duplicate ACKs:
cwnd only halved

Which loss was B?
a timeout: cwnd went
all the way to 1

The shape of the drop tells you which of the two events happened.

Ten parts, one line each

- | | | |
|----|-------------------|--|
| 01 | Congestion | More in than out, for long enough that the queue does not drain. |
| 02 | Causes | One sender too fast for its path, or many senders together. |
| 03 | Challenges | A mesh of shared links, no coordination, and a right rate that keeps moving. |
| 04 | The costs | Past capacity, load turns into waiting, repeats and wasted hops. |
| 05 | The rate | Aim for the bottleneck's rate; the lever is cwnd: $\text{rate} = \text{cwnd} \times \text{MTU} / \text{RTT}$. |
| 06 | Probing | Nothing tells the sender the rate: climb until a loss, back off, climb again. |
| 07 | AIMD | Add one when nothing is lost, halve on a loss: two senders end up equal. |
| 08 | Throughput | About $1.22 \cdot \text{MTU} / (\text{RTT} \cdot \sqrt{p})$: the loss rate sets the rate. |
| 09 | Slow start | Double every round trip up to ssthresh, then add one. |
| 10 | On a loss | Three duplicate ACKs: halve. A timeout: back to one. |

Each part answered one question. Together they make up classic TCP congestion control.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

One more row in the table

the defect	the mechanism it forces
⚡ Bits flip inside a packet	Checksum
☑ A cannot see what arrived at B	ACK from the receiver
# The feedback itself is damaged	Sequence number
🕒 A packet is lost	Timer; resend when it expires
↩ Loss looks the same as delay	Adaptive timeout, doubled after expiry
🔄 The link sits idle	Window of bandwidth × RTT packets
🔄 One packet in a window is lost	Go-Back-N or selective repeat
🔄 A number is reused too soon	Window ≤ half the sequence space
📄 Loss is detected late	Duplicate ACKs, selective ACK
📺 The receiver overflows	not covered in this course
📺 The network between them overflows	Congestion window, AIMD

Row 10 is a real defect with a real mechanism, the advertised window. This course does not spend a lecture on it.

Every mechanism here exists because the protocol before it broke in one way.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss

Three assumptions this design makes



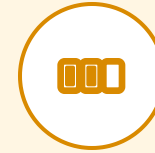
A loss means congestion

what about a wireless link that loses packets with an empty buffer?



Halving is right

what happens on a 10 Gbps path, where the window is 83,333 packets?



A full buffer is acceptable

what does filling every buffer before reacting do to everyone else's delay?

Each of the three is a design decision, not a law of nature.

The next two lectures replace each of them, and many of today's senders already do.

AIMD is one point in a design space. The next lecture walks to the other points.

1 Congestion 2 Causes 3 Challenges 4 The costs 5 The rate 6 Probing 7 AIMD 8 Throughput 9 Slow start 10 On a loss